Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

# Raport

## la lucrarea de laborator nr. 5

Disciplina: Tehnici de programare

### Tema: Funcții, metode și importuri în Python pentru web

A efectuat: Moraru Elena, grupa SI-264

A verificat: asistent universitar Reițman Patricia

Chișinău 2026

# Scopul lucrării și organizarea raportului

Am realizat exercițiile laboratorului 5 pentru a exersa importurile, metodele, atributele și funcțiile Python în comunicarea cu servicii web. Raportul prezintă fiecare exercițiu separat, cu o captură reală a codului deschis în Visual Studio Code, o explicație și rezultatul rulării.

Capturile sunt numerotate Fig. 1.1 până la Fig. 1.50. Pentru implementările care depășesc zona vizibilă a editorului sunt incluse continuări. Fișierele individuale ex01.py până la ex50.py se află în directorul exercises, iar ieșirile complete sunt salvate în logs/exercises.

### Mediul de lucru

Python 3.14.7, requests 2.34.2 și BeautifulSoup 4.14.3. Cererile de citire sunt adresate cybercor.org; antetele personalizate sunt verificate pe httpbin.org. Timeout-ul este 10 secunde, cu o pauză de o secundă între cererile repetate.

```
python main.py https://cybercor.org
python -m unittest -v test_webtools
```

Rulările au fost efectuate la 13 octombrie 2026. Datele DNS, antetele și certificatul descriu momentul verificării. Un răspuns 200 nu dovedește singur existența unei pagini distincte, iar Requests nu execută JavaScript.

### Structura lucrării

Partea 1 Importuri Exercițiile 1 până la 8

Partea 2 Metode și atribute Exercițiile 9 până la 20

Partea 3 Funcții proprii Exercițiile 21 până la 34

Partea 4 Module standard Exercițiile 35 până la 43

Partea 5 Modul propriu și proiect final Exercițiile 44 până la 50

# Partea 1 Importuri

### Exercițiul 1 Primul import

Am importat biblioteca externă requests și modulul standard urllib.request. Numai requests necesită instalare separată.

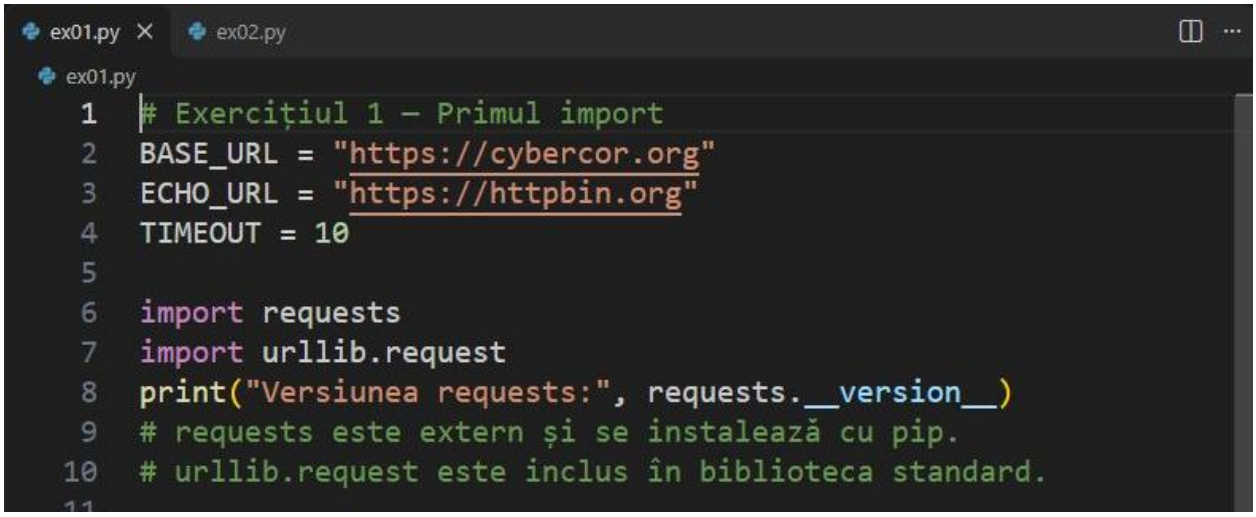



***Fig. 1.1. Codul exercițiului 1 în Visual Studio Code***

**Rezultatul rulării**

```
Versiunea requests: 2.34.2
```

### Exercițiul 2 Două stiluri de import

Am comparat importul modulului cu importul unei funcții. Ambele forme produc o cerere GET, dar numele calificat arată explicit biblioteca.

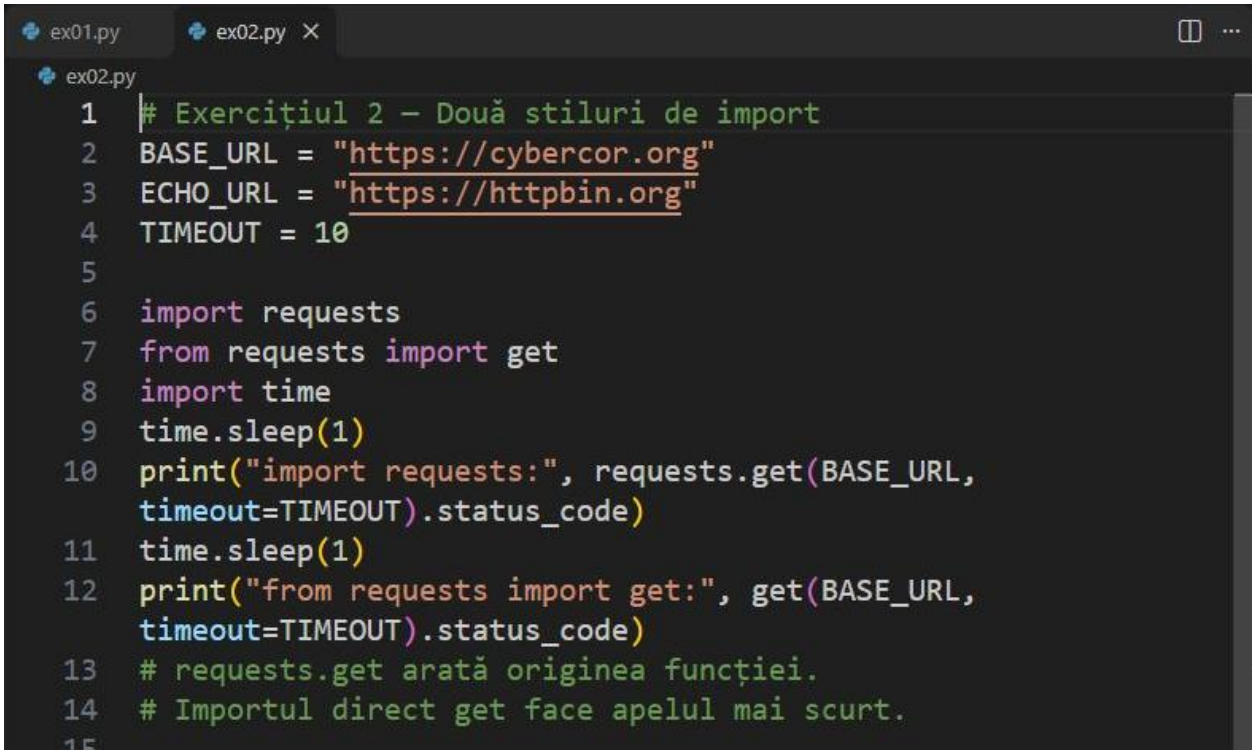



***Fig. 1.2. Codul exercițiului 2 în Visual Studio Code***

**Rezultatul rulării**

```
import requests: 200
from requests import get: 200
```

### Exercițiul 3 Alias pentru modul

Am folosit aliasul rq pentru requests și am păstrat timeout-ul cerut.

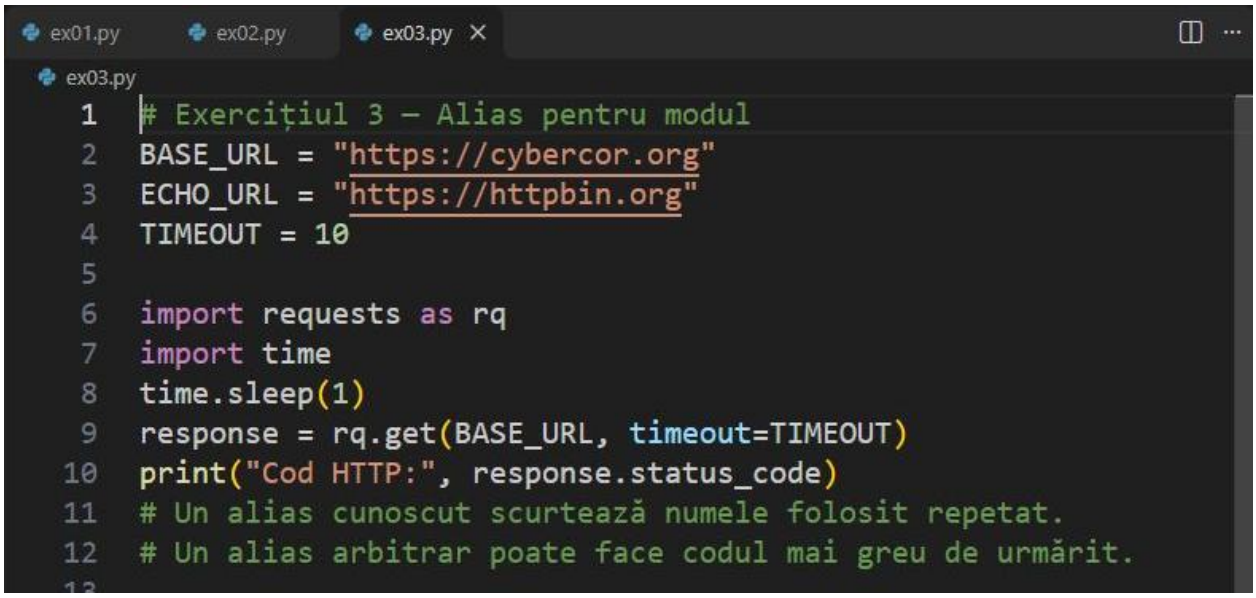



***Fig. 1.3. Codul exercițiului 3 în Visual Studio Code***

**Rezultatul rulării**

```
Cod HTTP: 200
```

### Exercițiul 4 Descărcare cu biblioteca standard

Am descărcat pagina cu urllib.request și am transformat octeții în text înainte de a afișa primele 200 de caractere.

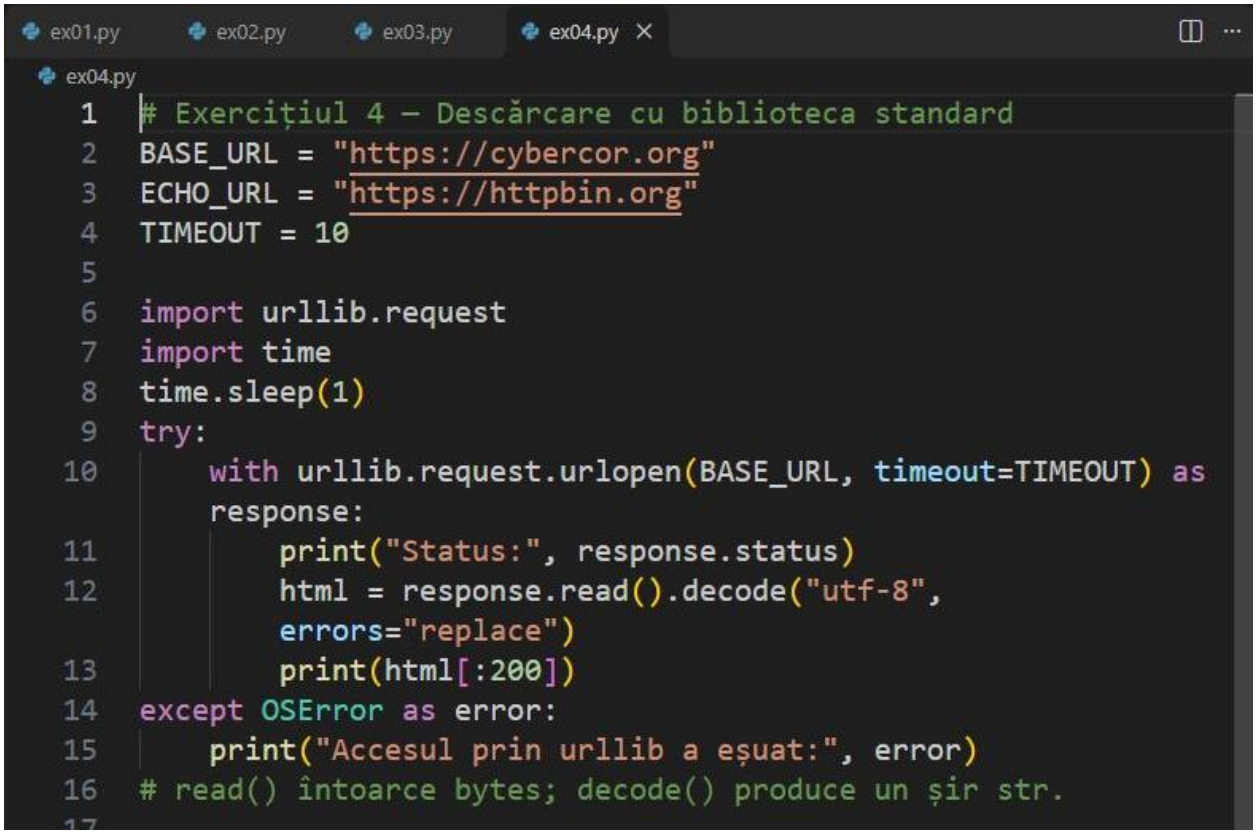



*Fig. 1.4. Codul exercițiului 4 în Visual Studio Code*

**Rezultatul rulării**

```
Accesul prin urllib a eșuat: HTTP Error 403: Forbidden
```

În această rulare, serverul a refuzat cererea urllib cu status 403. Excepția este tratată și afișată; programul continuă. Codul pentru citire și decodare se execută atunci când serverul permite accesul.

### Exercițiul 5 Explorarea modulului requests

dir oferă numele expuse de modul. Am identificat o funcție, o clasă și un modul.




*Fig. 1.5. Codul exercițiului 5 în Visual Studio Code*

**Rezultatul rulării**

`dir(requests) a afișat numele modulului; printre ele: get, Session, exceptions.`

### Exercițiul 6 Documentația funcției get

Am consultat help și am folosit parametrul timeout. Acesta nu este un termen absolut pentru întreaga descărcare.

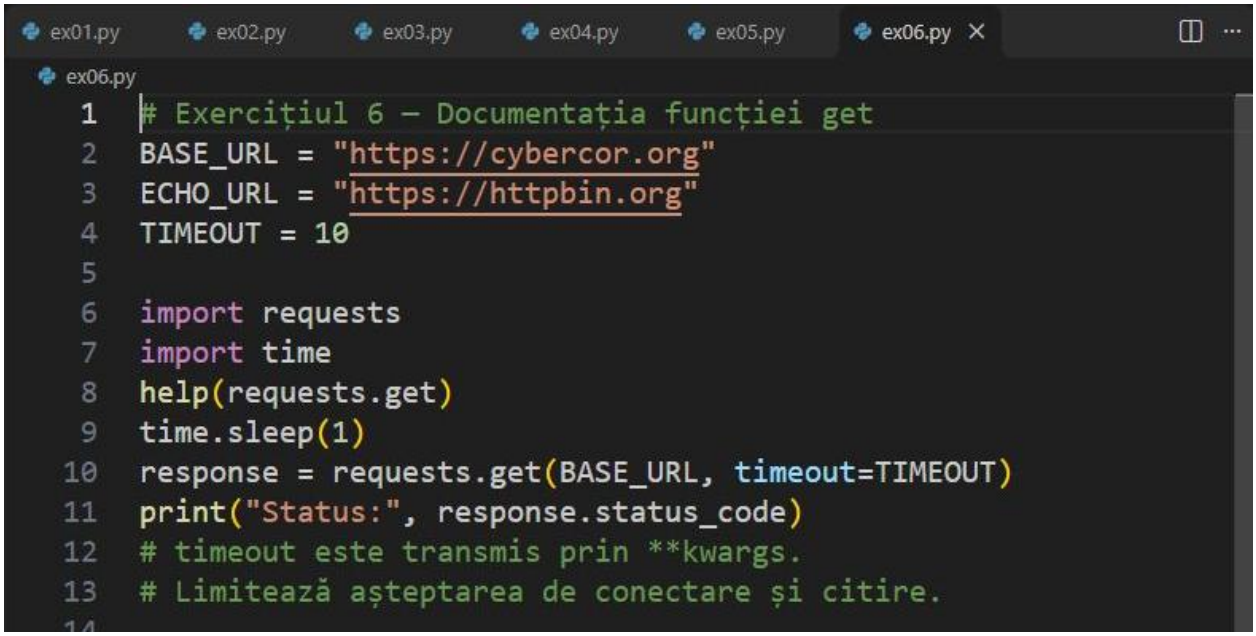



*Fig. 1.6. Codul exercițiului 6 în Visual Studio Code*

**Rezultatul rulării**

```
Status: 200
```

### Exercițiul 7 Cronometrarea unei cereri

Am comparat durata completă măsurată cu perf_counter cu intervalul response.elapsed.

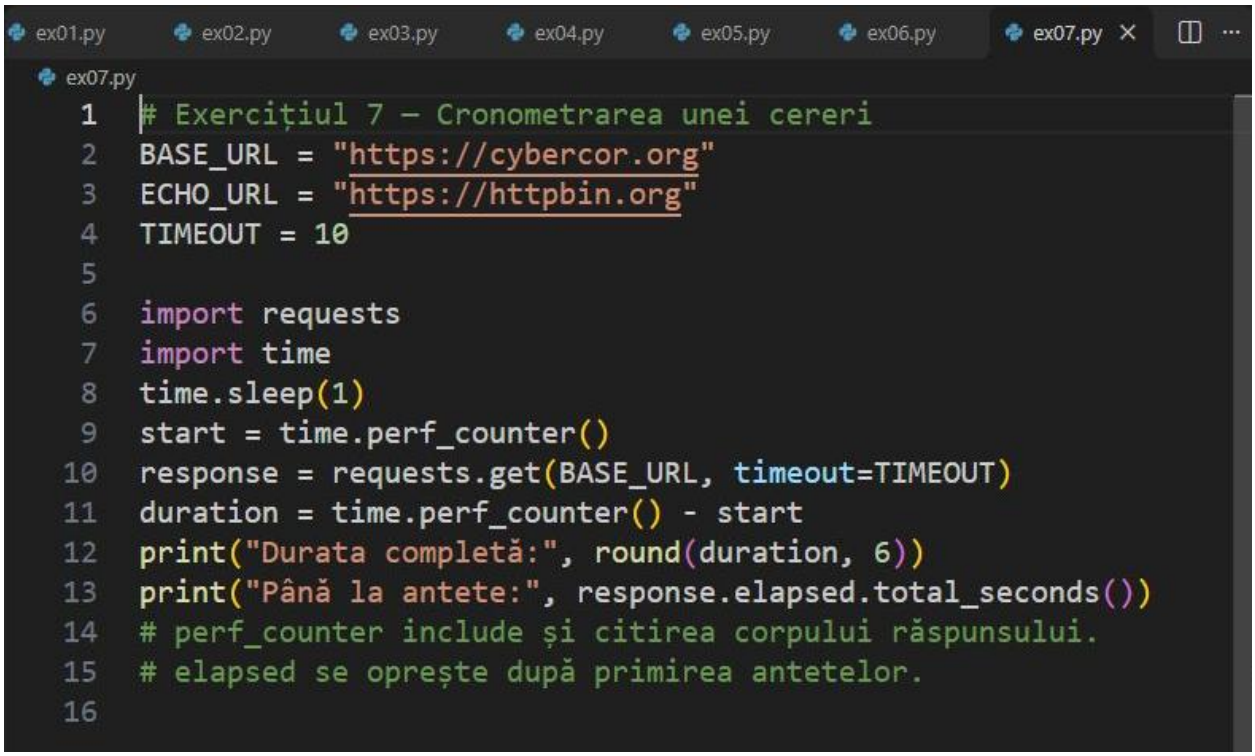



*Fig. 1.7. Codul exercițiului 7 în Visual Studio Code*

**Rezultatul rulării**

```
Durata completă: 0.291528
Până la antete: 0.285326
```

### Exercițiul 8 Tratarea unui import absent

Am tratat ImportError și am prevăzut un mesaj de instalare. În mediul folosit, BeautifulSoup este deja instalat.

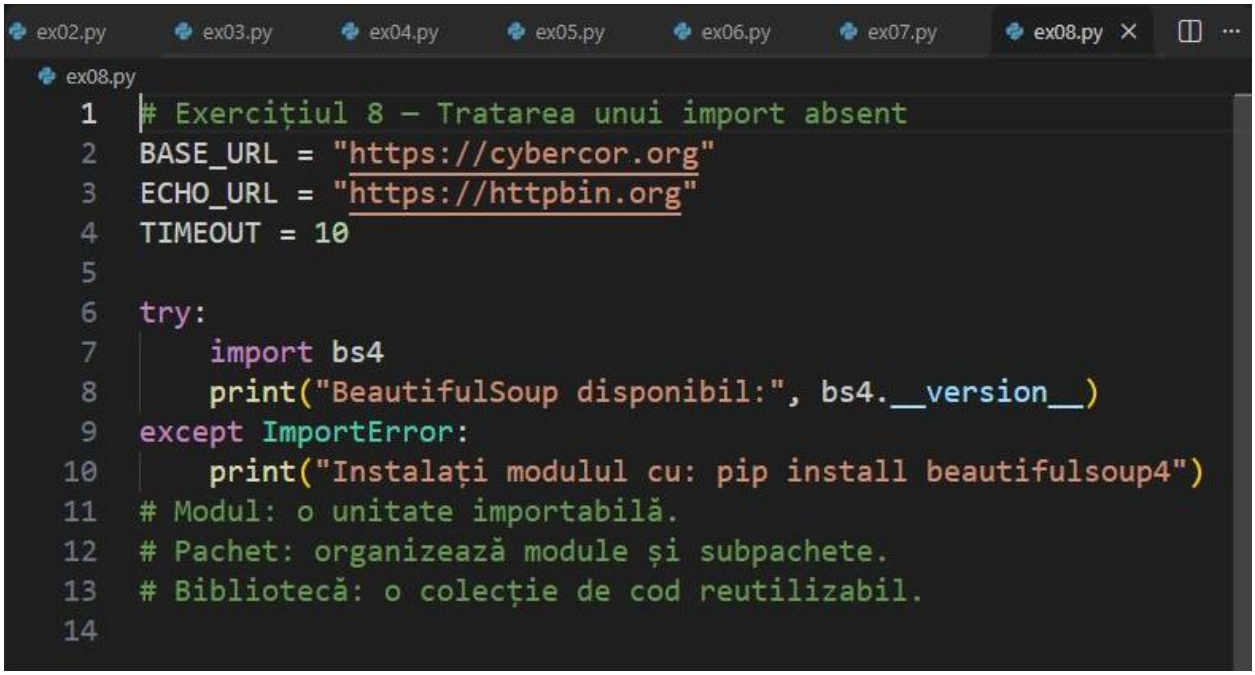



***Fig. 1.8. Codul exercițiului 8 în Visual Studio Code***

**Rezultatul rulării**

```
BeautifulSoup disponibil: 4.14.3
```

# Partea 2 Metode și atribute

### Exercițiul 9 Atributele răspunsului

Am afișat patru atribute ale răspunsului HTTP. Acestea se citesc fără paranteze.

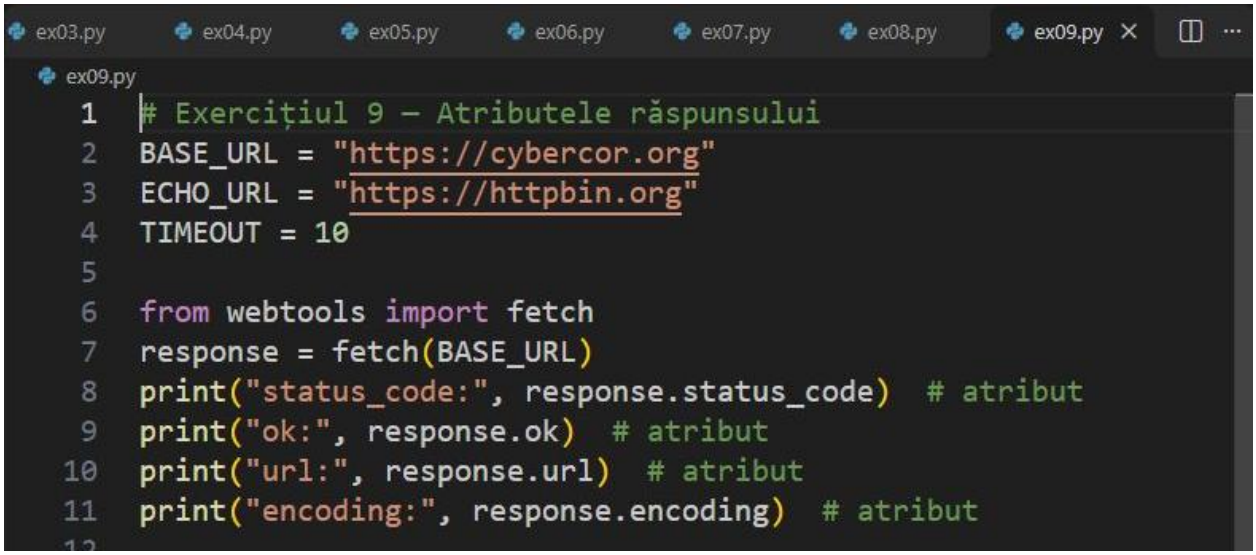



***Fig. 1.9. Codul exercițiului 9 în Visual Studio Code***

**Rezultatul rulării**

```
status_code: 200
ok: True
url: https://cybercor.org/
encoding: utf-8
```

### Exercițiul 10 Tratarea erorilor HTTP

Am apelat raise_for_status și am tratat HTTPError. Răspunsul real pentru calea de test poate fi 200, caz în care metoda nu ridică excepția.

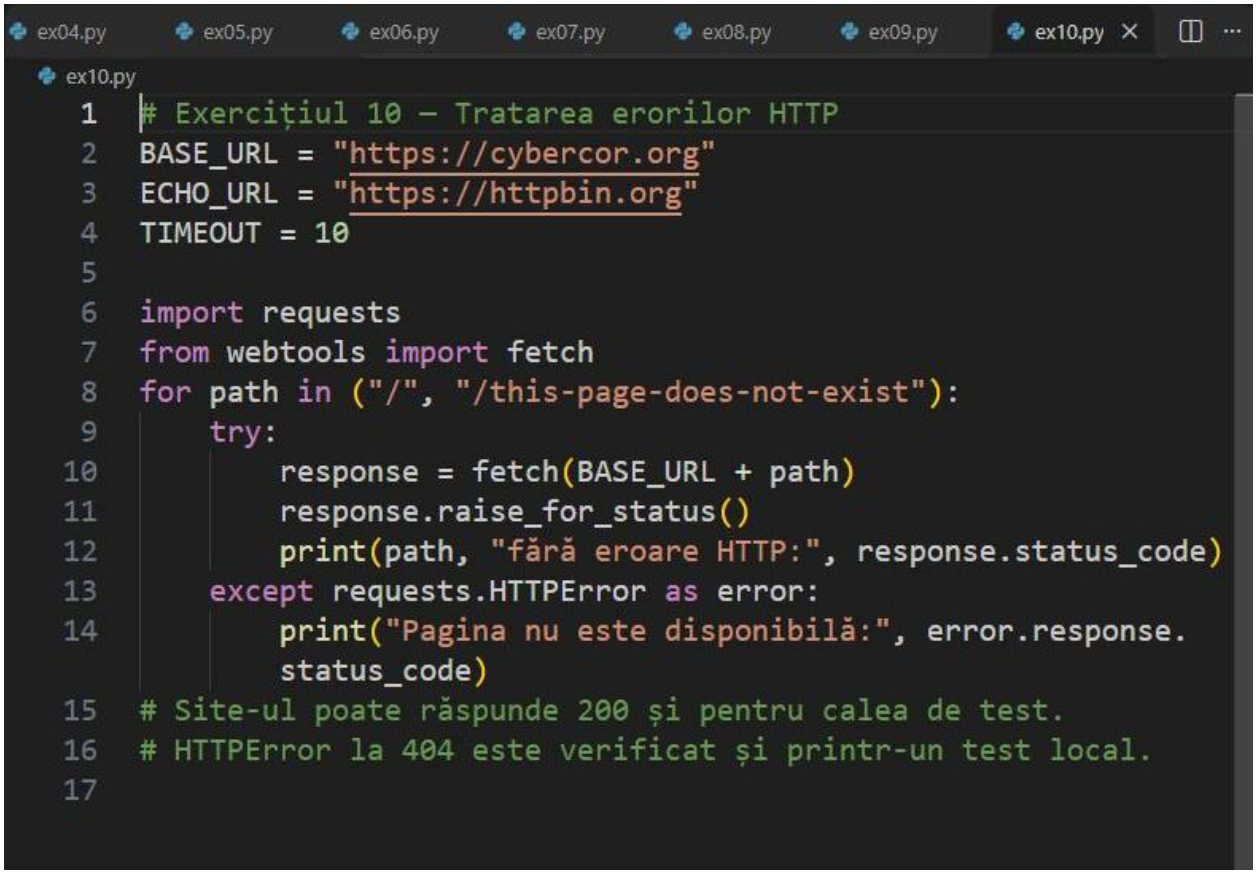



***Fig. 1.10. Codul exercițiului 10 în Visual Studio Code***

**Rezultatul rulării**

```
/ fără eroare HTTP: 200
/this-page-does-not-exist fără eroare HTTP: 200
```

Ambele căi au întors 200 în rularea reală. De aceea nu s-a ridicat HTTPError. Comportamentul la 404 este verificat separat printr-un răspuns construit local în test_webtools.py.

### Exercițiul 11 Afișarea tuturor antetelor

Am parcurs antetele prin metoda items și am afișat fiecare pereche nume valoare.

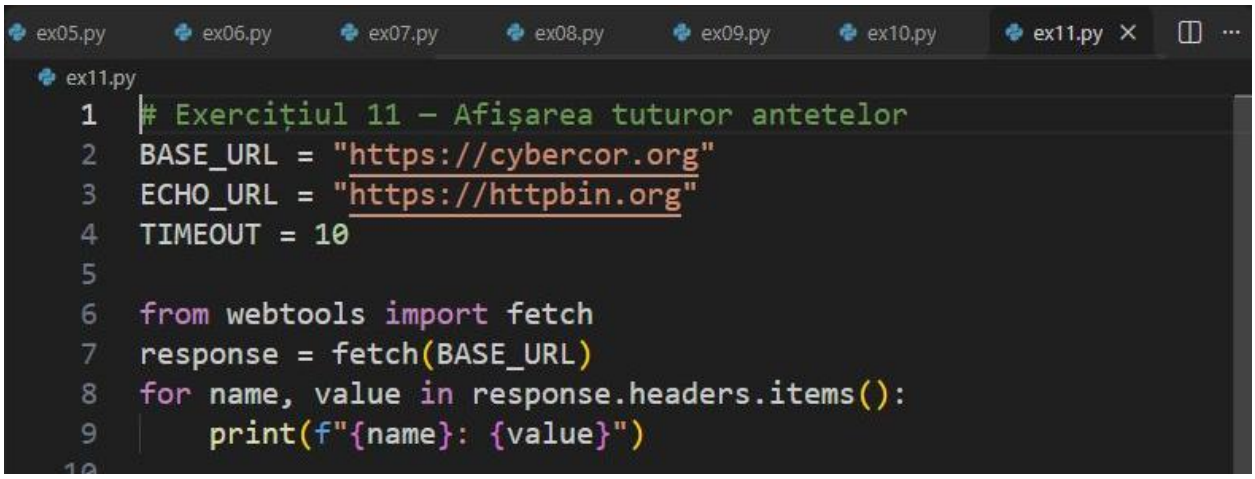



*Fig. 1.11. Codul exercițiului 11 în Visual Studio Code*

**Rezultatul rulării**

```
Content-Type: text/html; charset=utf-8
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
Server: cloudflare
```

### Exercițiul 12 Citirea antetelor cu get

Am folosit o valoare implicită pentru antetele absente și am verificat că numele lor nu disting majusculele.

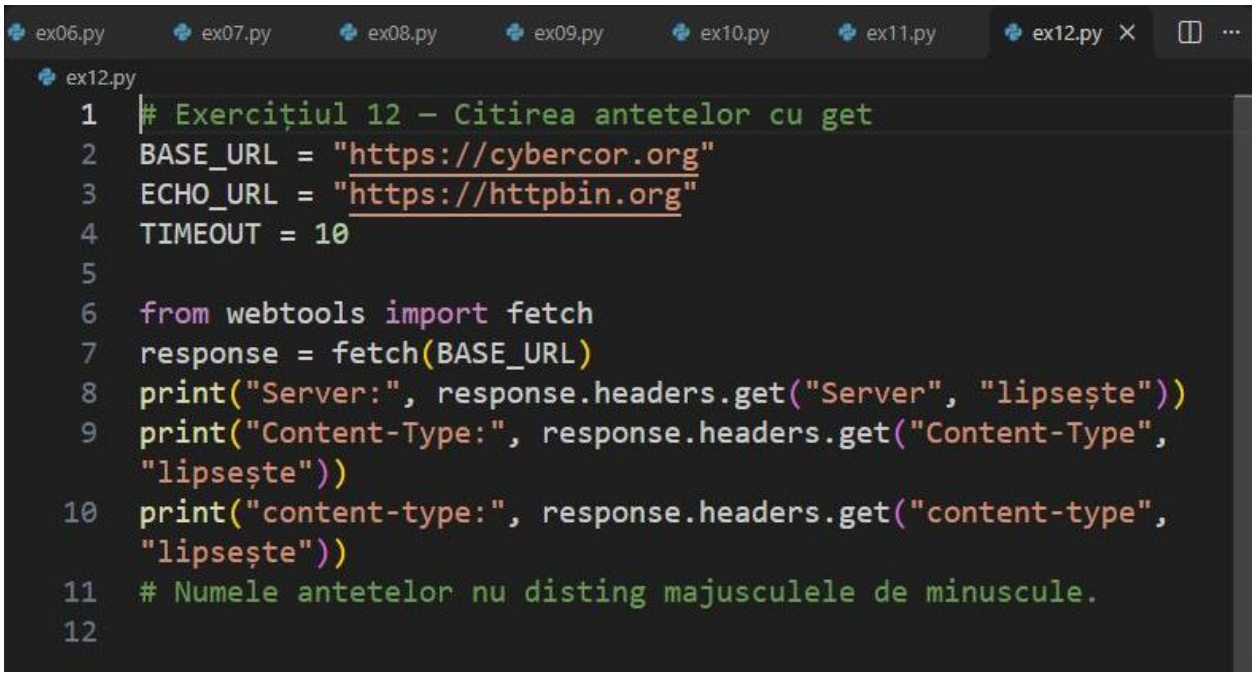



*Fig. 1.12. Codul exercițiului 12 în Visual Studio Code*

**Rezultatul rulării**

```
Server: cloudflare
Content-Type: text/html; charset=utf-8
content-type: text/html; charset=utf-8
```

### Exercițiul 13 Înlănțuirea metodelor de șir

Am numărat aparițiile șirului cyber după transformarea textului în litere mici.

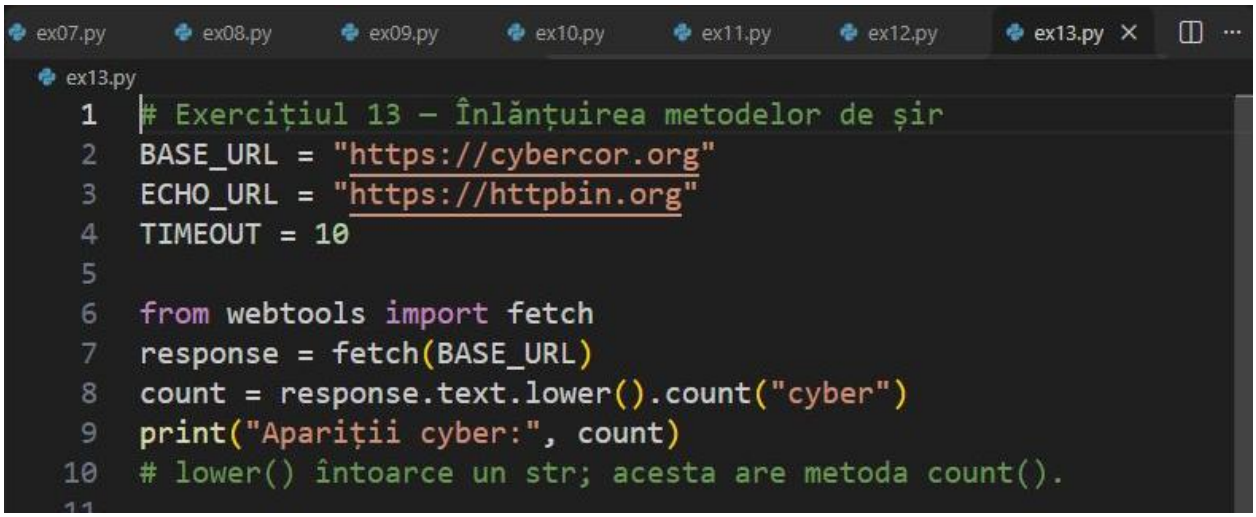



*Fig. 1.13. Codul exercițiului 13 în Visual Studio Code*

**Rezultatul rulării**

```
Apariții cyber: 9
```

### Exercițiul 14 Extragerea titlului cu find

Am localizat tag-urile title cu find, am extras conținutul prin slicing și am eliminat spațiile exterioare.

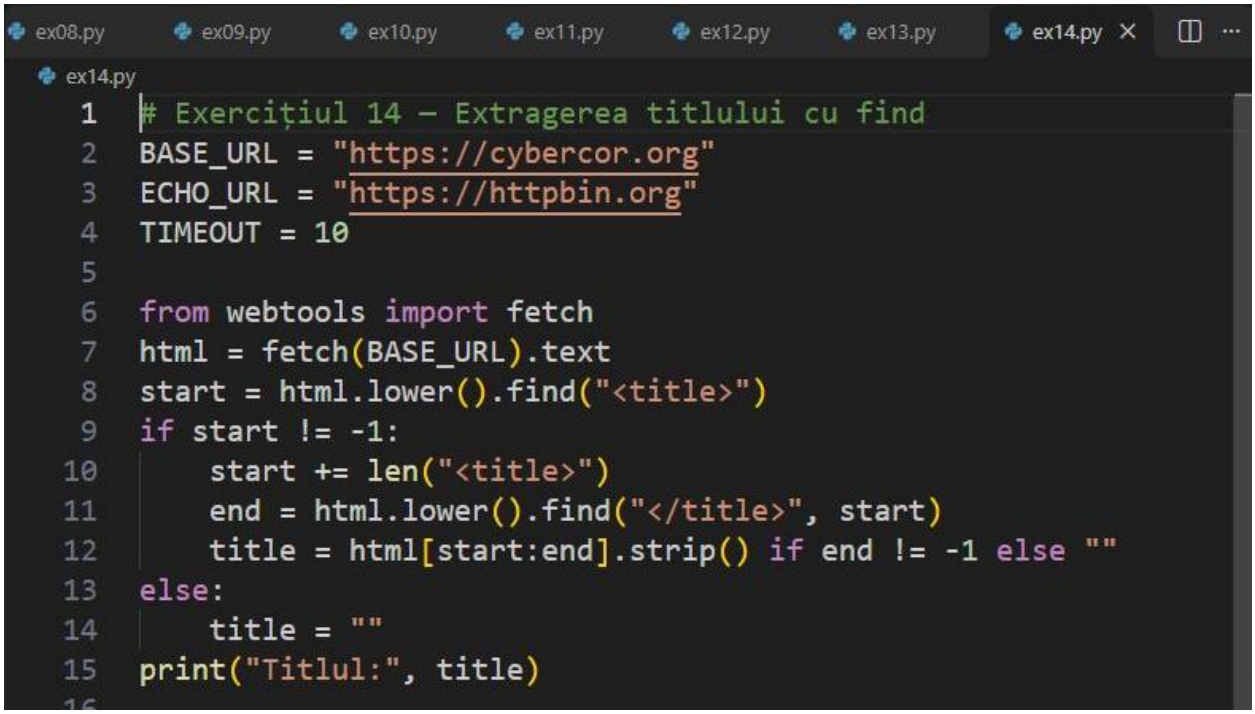



*Fig. 1.14. Codul exercițiului 14 în Visual Studio Code*

**Rezultatul rulării**

```
Titlul: CYBERCOR
```

### Exercițiul 15 Numărarea liniilor HTML

Am folosit splitlines și max cu key=len. Valoarea implicită permite tratarea unui corp gol.

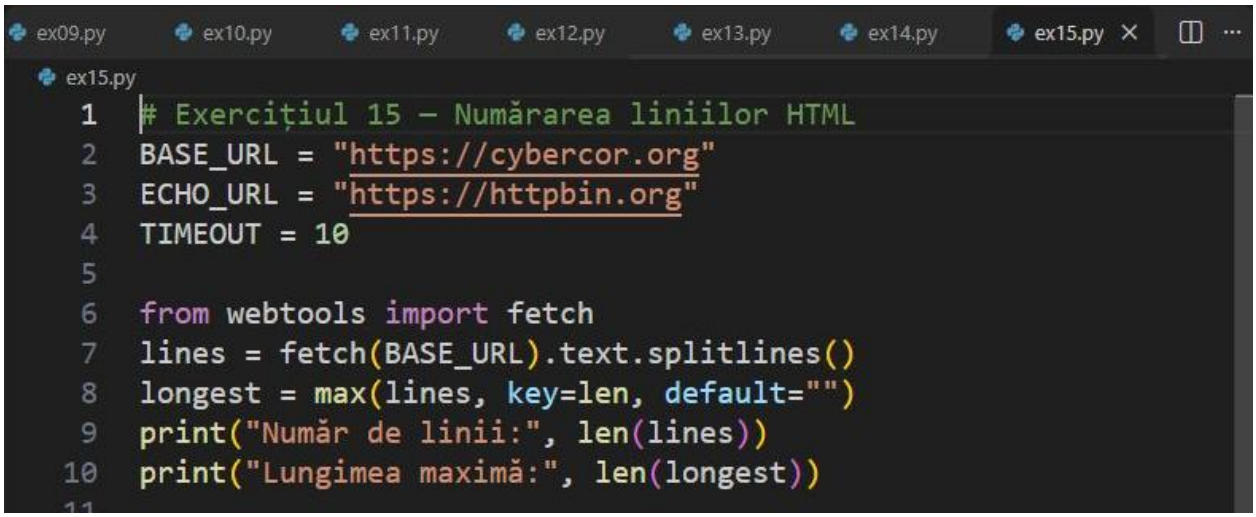



***Fig. 1.15. Codul exercițiului 15 în Visual Studio Code***

**Rezultatul rulării**

```
Număr de linii: 32
Lungimea maximă: 218
```

### Exercițiul 16 Verificarea HTTPS

Am verificat prefixul URL-ului final, după eventualele redirecționări.

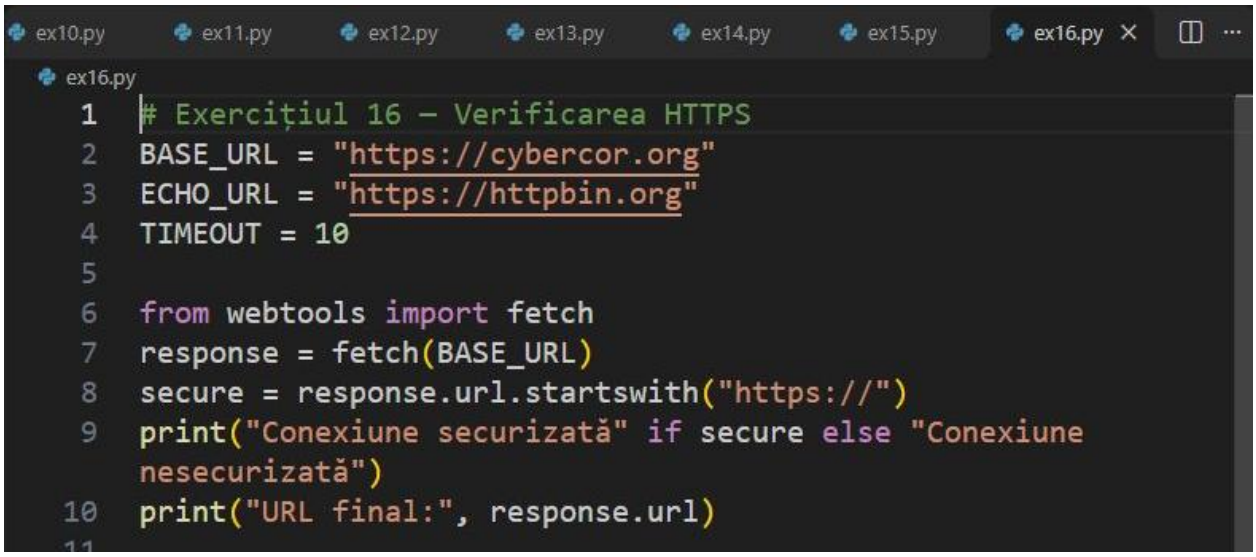



***Fig. 1.16. Codul exercițiului 16 în Visual Studio Code***

**Rezultatul rulării**

```
Conexiune securizată
URL final: https://cybercor.org/
```

### Exercițiul 17 Lanțul de redirecționări

Am pornit de la HTTP și am parcurs istoricul Requests până la URL-ul final.

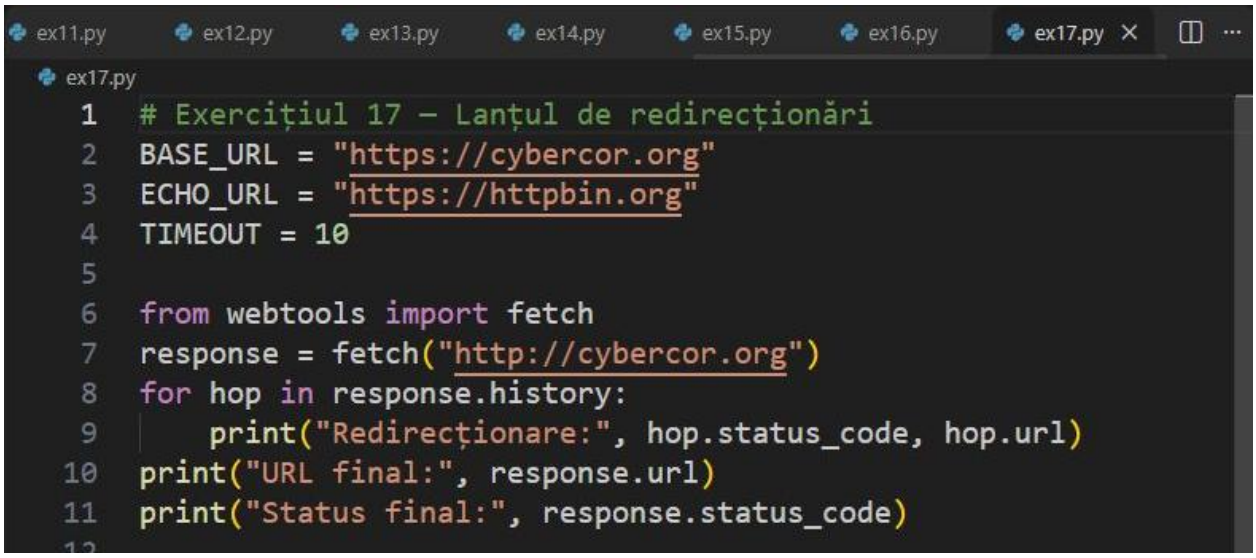



***Fig. 1.17. Codul exercițiului 17 în Visual Studio Code***

**Rezultatul rulării**

```
Redirecționare: 301 http://cybercor.org/
URL final: https://cybercor.org/
Status final: 200
```

### Exercițiul 18 Comparația HEAD cu GET

Am comparat lungimea corpurilor primite prin HEAD și GET, folosind o sesiune cu pauze între cereri.

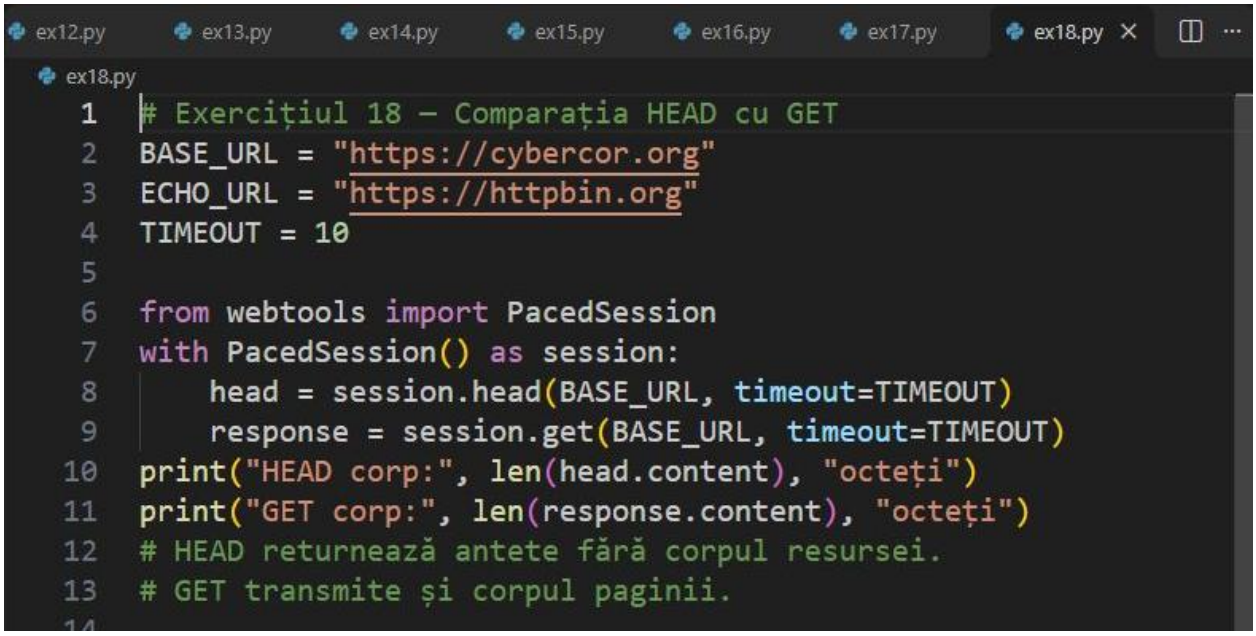



***Fig. 1.18. Codul exercițiului 18 în Visual Studio Code***

**Rezultatul rulării**

```
HEAD corp: 0 octeți
GET corp: 1540 octeți
```

### Exercițiul 19 Cookie-urile răspunsului

Am parcurs cookie-urile setate în răspuns și am citit atributele name și secure.

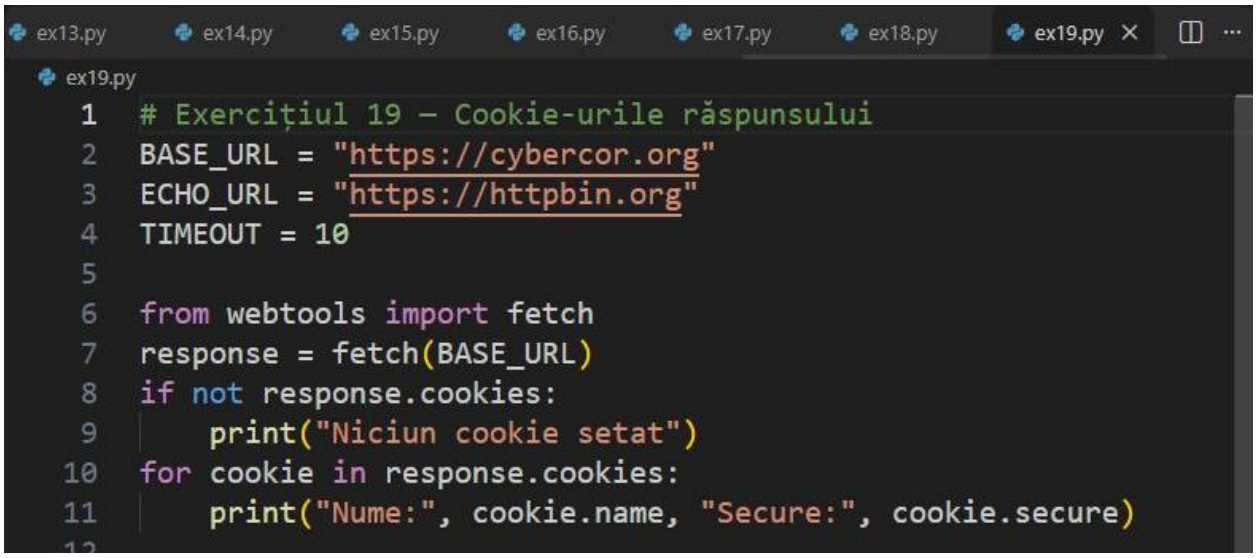



***Fig. 1.19. Codul exercițiului 19 în Visual Studio Code***

**Rezultatul rulării**

```
Niciun cookie setat
```

### Exercițiul 20 Sesiune cu User Agent personalizat

Am trimis antetul personalizat numai la httpbin și am comparat valoarea întoarsă cu cea configurată.

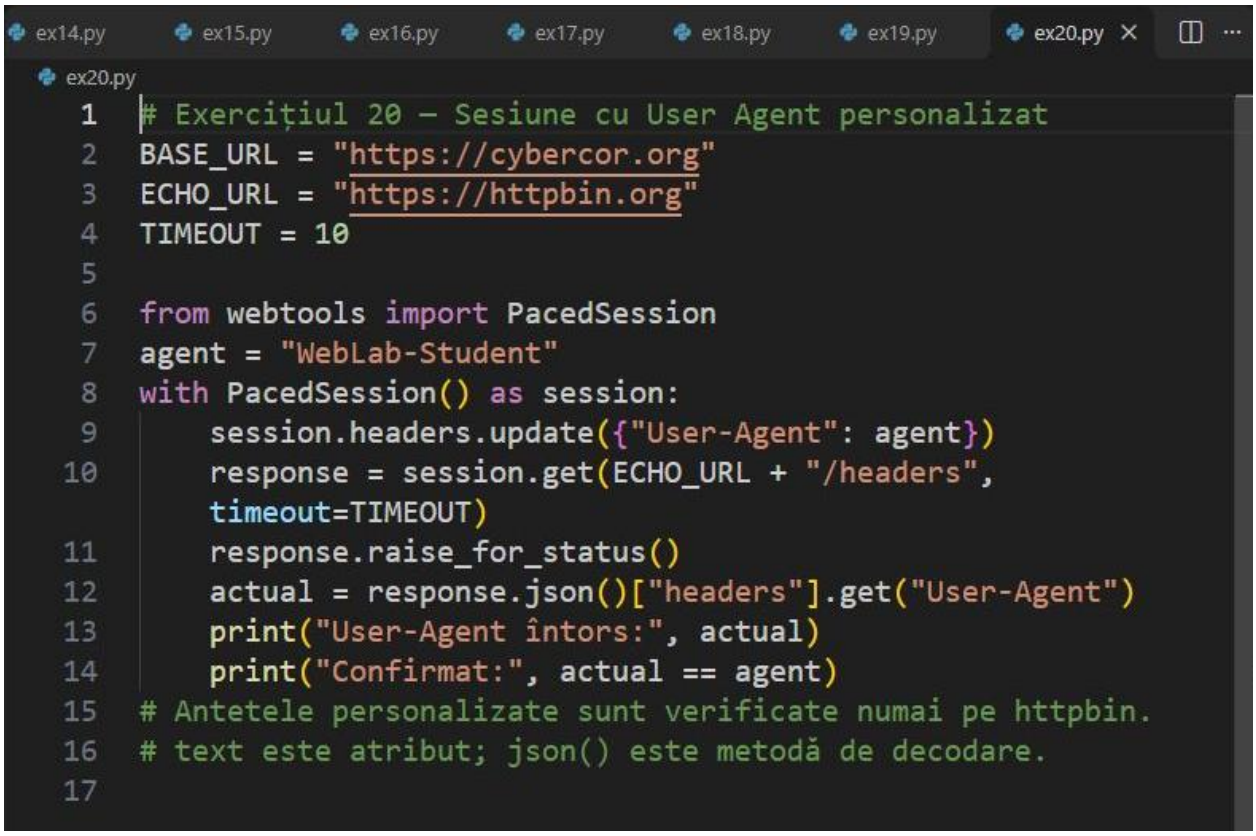



*Fig. 1.20. Codul exercițiului 20 în Visual Studio Code*

**Rezultatul rulării**

```
User-Agent întors: WebLab-Student
Confirmat: True
```

# Partea 3 Funcții proprii

### Exercițiul 21 Prima funcție fetch

Am separat cererea GET într-o funcție care returnează obiectul Response.

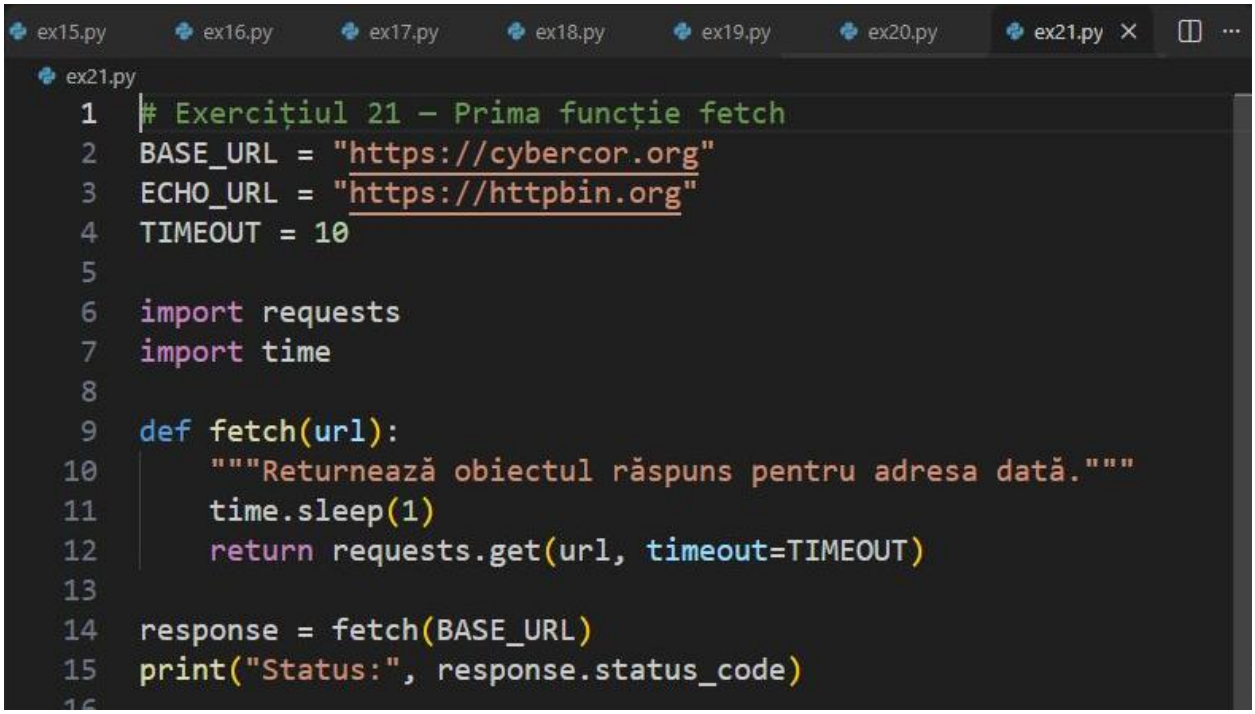



*Fig. 1.21. Codul exercițiului 21 în Visual Studio Code*

**Rezultatul rulării**

```
Status: 200
```

### Exercițiul 22 Returnarea codului HTTP

Am returnat o singură valoare, codul HTTP, și am verificat cele trei căi cerute.

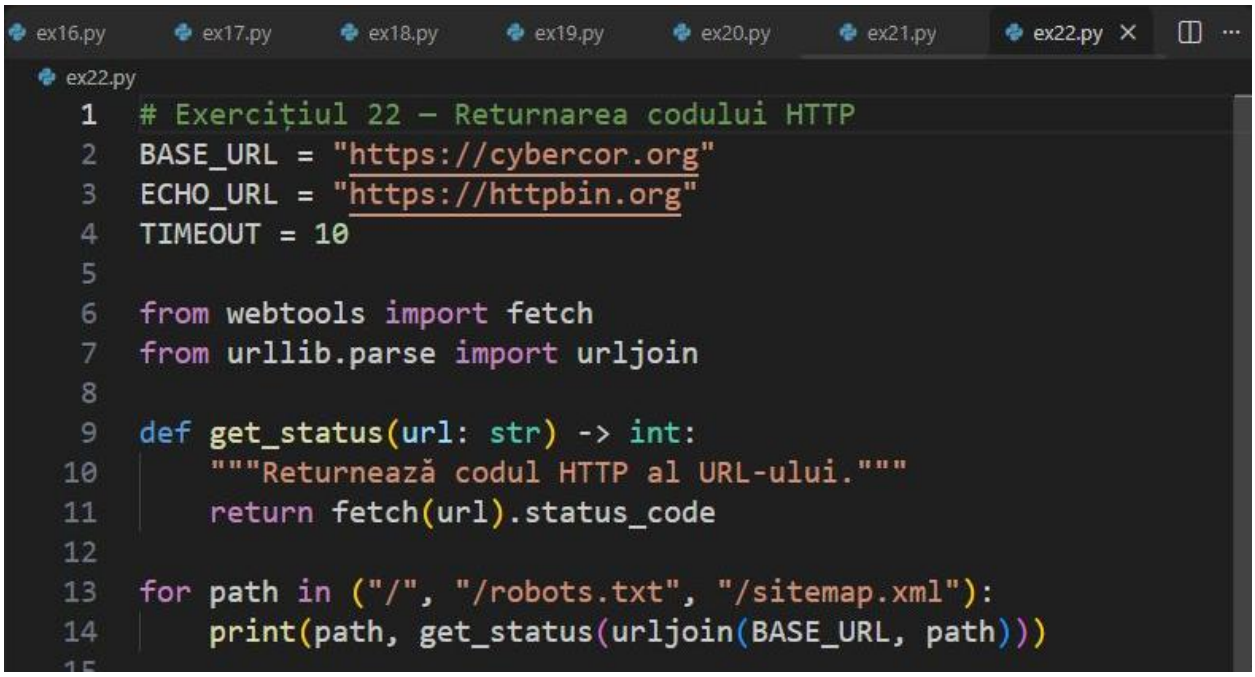



***Fig. 1.22. Codul exercițiului 22 în Visual Studio Code***

**Rezultatul rulării**

```
/ 200
/robots.txt 200
/sitemap.xml 200
```

### Exercițiul 23 Parametri cu valori implicite

Am apelat funcția o dată cu timeout implicit și o dată cu timeout=3.

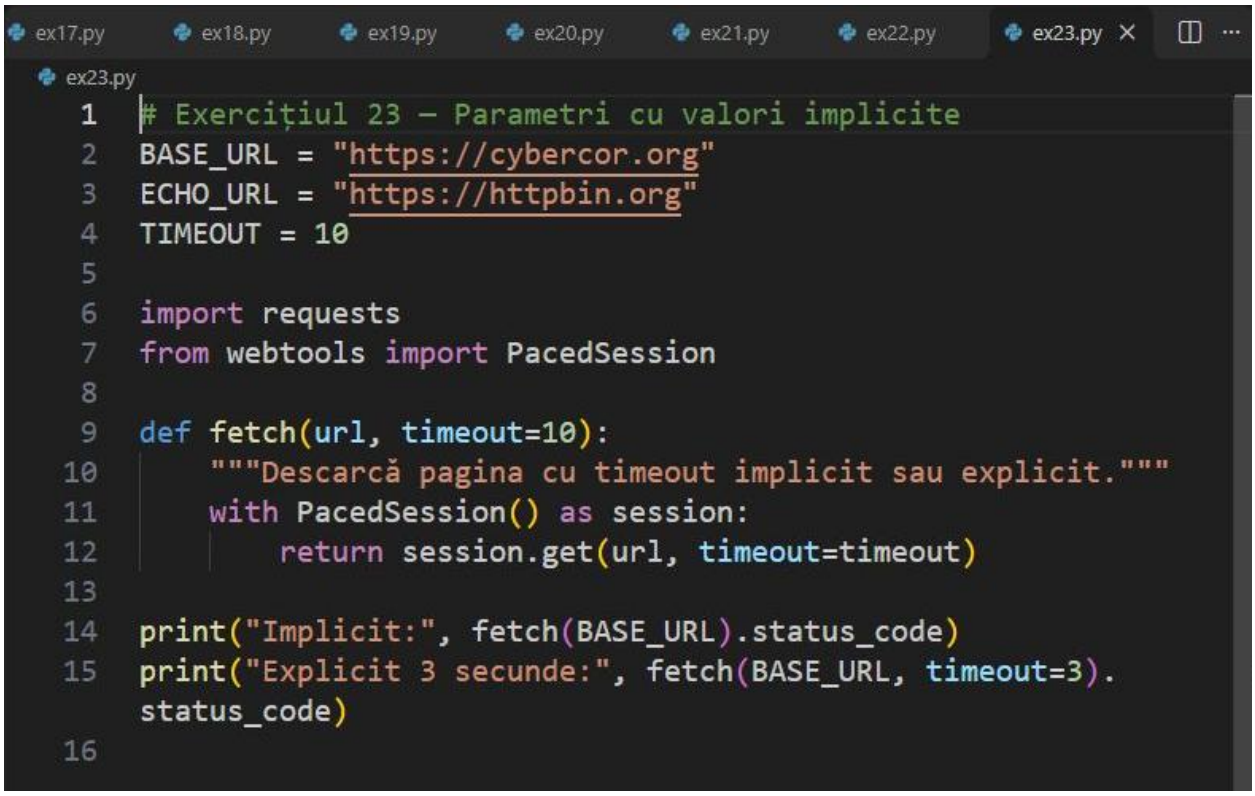



***Fig. 1.23. Codul exercițiului 23 în Visual Studio Code***

**Rezultatul rulării**

```
Implicit: 200
Explicit 3 secunde: 200
```

### Exercițiul 24 Separarea descărcării de analiză

get_title primește HTML și nu efectuează cereri. Această separare permite teste locale.

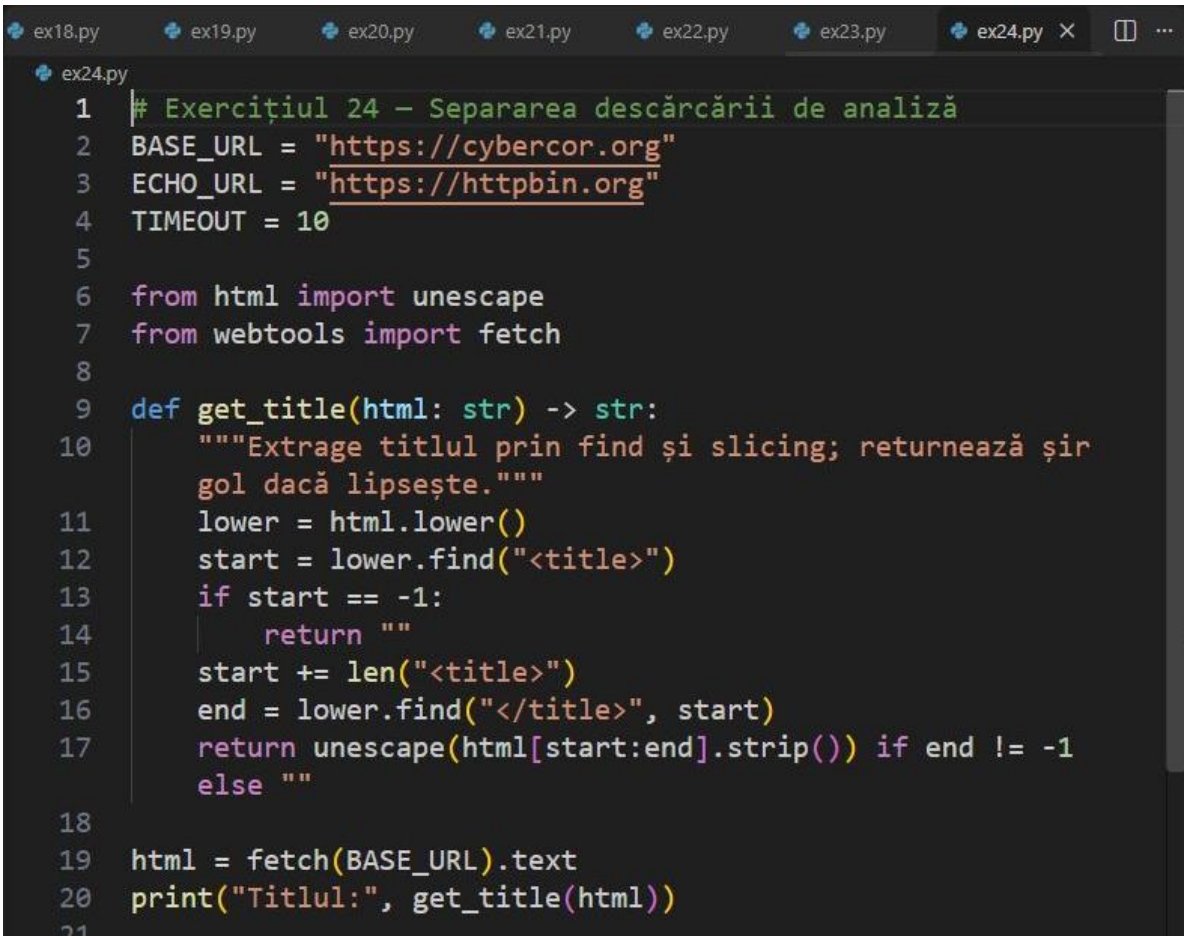



*Fig. 1.24. Codul exercițiului 24 în Visual Studio Code*

**Rezultatul rulării**

```
Titlul: CYBERCOR
```

### Exercițiul 25 Docstring și help

Am adăugat un docstring și l-am afișat prin help.

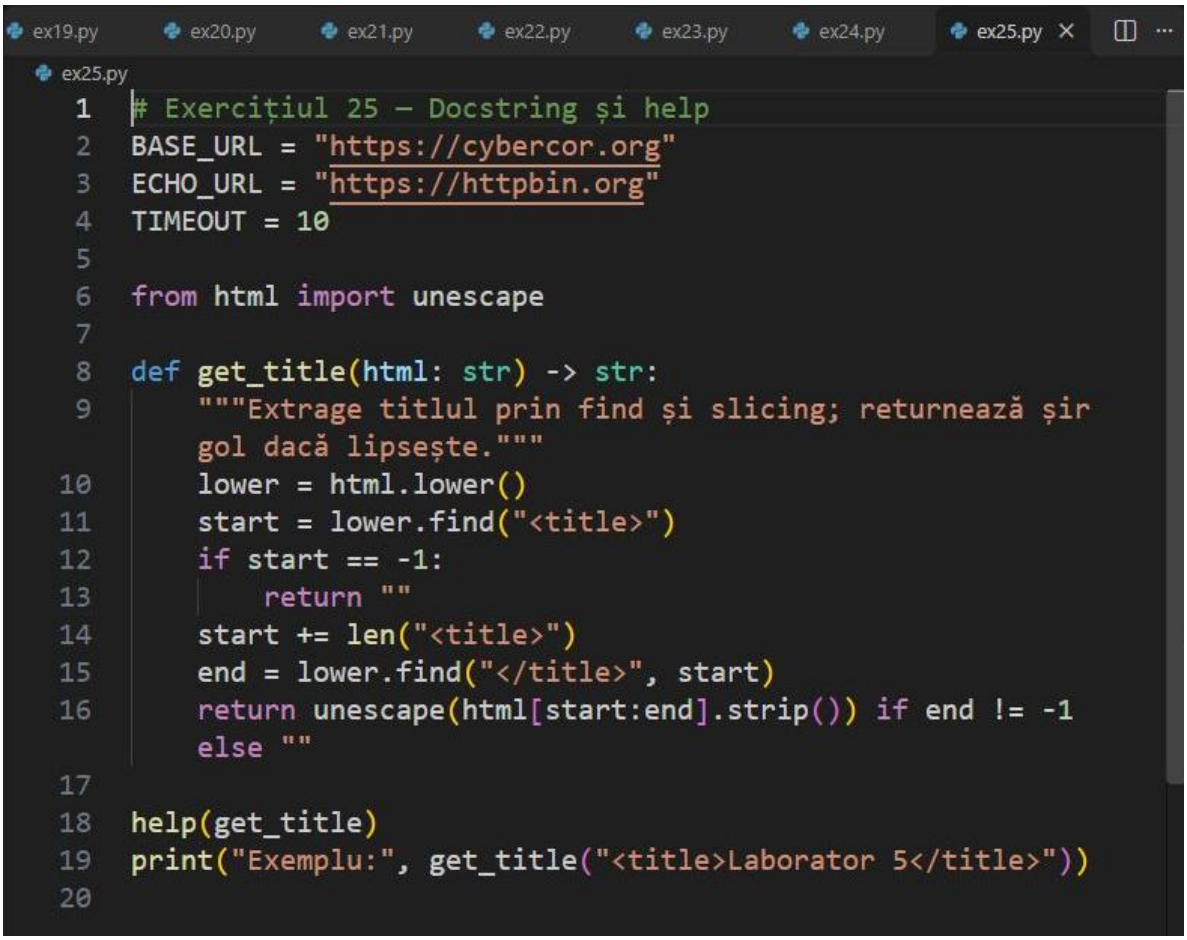



*Fig. 1.25. Codul exercițiului 25 în Visual Studio Code*

**Rezultatul rulării**

```
Help on function get_title in module __main__:

get_title(html: str) -> str
    Extrage titlul prin find și slicing; returnează șir gol dacă lipsește.

Exemplu: Laborator 5
```

### Exercițiul 26 Adnotări de tip

Am demonstrat că adnotările de tip nu blochează apelul; validarea URL-ului se produce în Requests.

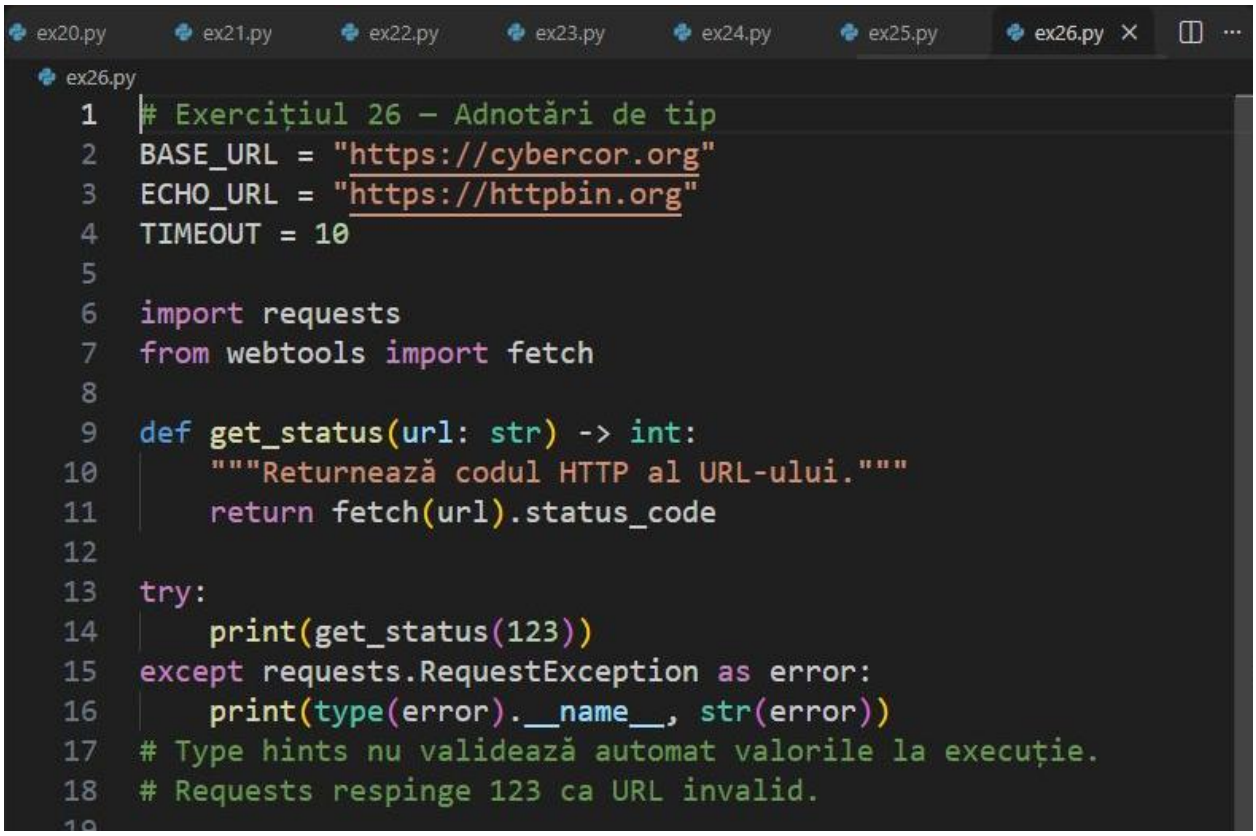



*Fig. 1.26. Codul exercițiului 26 în Visual Studio Code*

**Rezultatul rulării**

```
MissingSchema Invalid URL '123': No scheme supplied. Perhaps you meant https://123?
```

### Exercițiul 27 Funcția page_exists

Am tratat erorile de rețea astfel încât funcția să întoarcă False și să nu oprească programul.

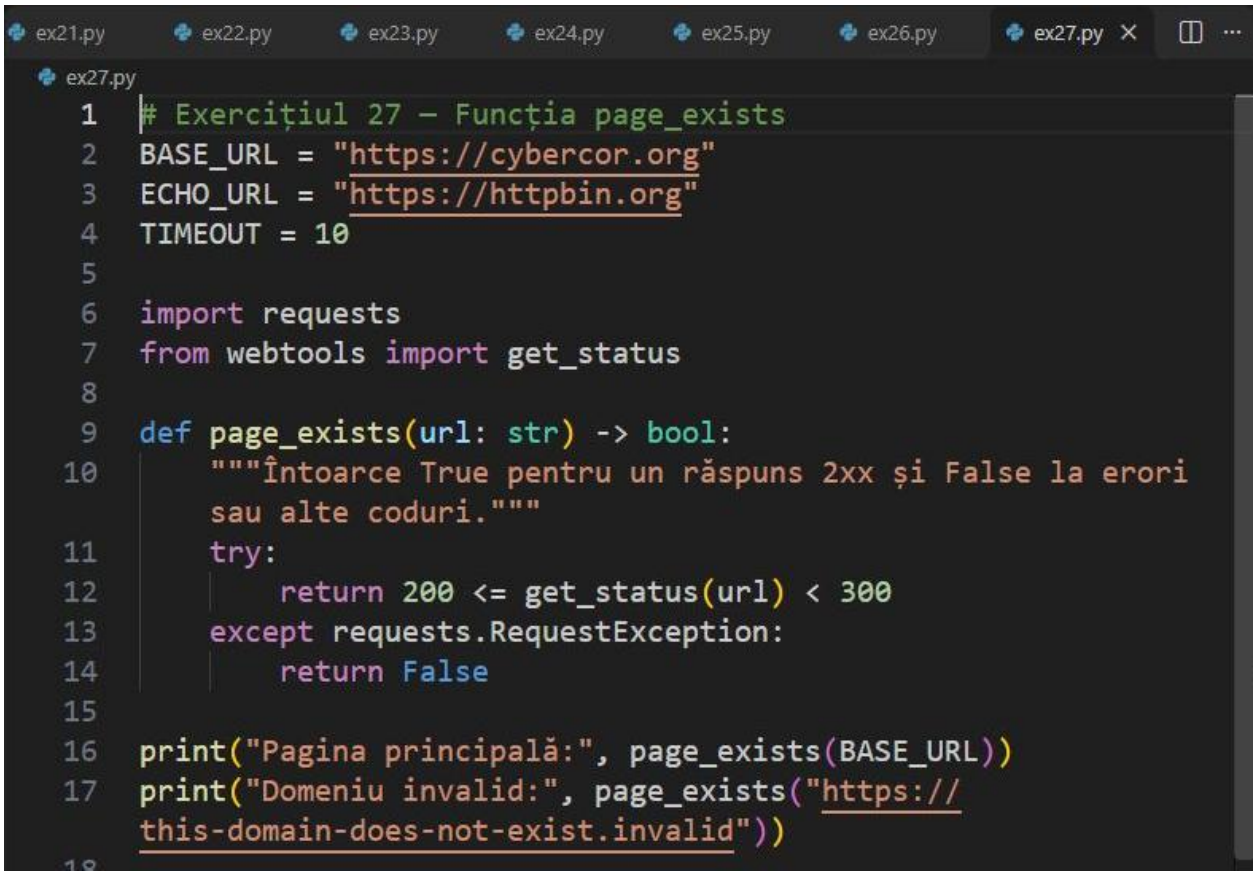



*Fig. 1.27. Codul exercițiului 27 în Visual Studio Code*

**Rezultatul rulării**

```
Pagina principală: True
Domeniu invalid: False
```

### Exercițiul 28 Verificarea unei liste de căi

Am construit un dicționar cale cod HTTP. None marchează o eroare de rețea.

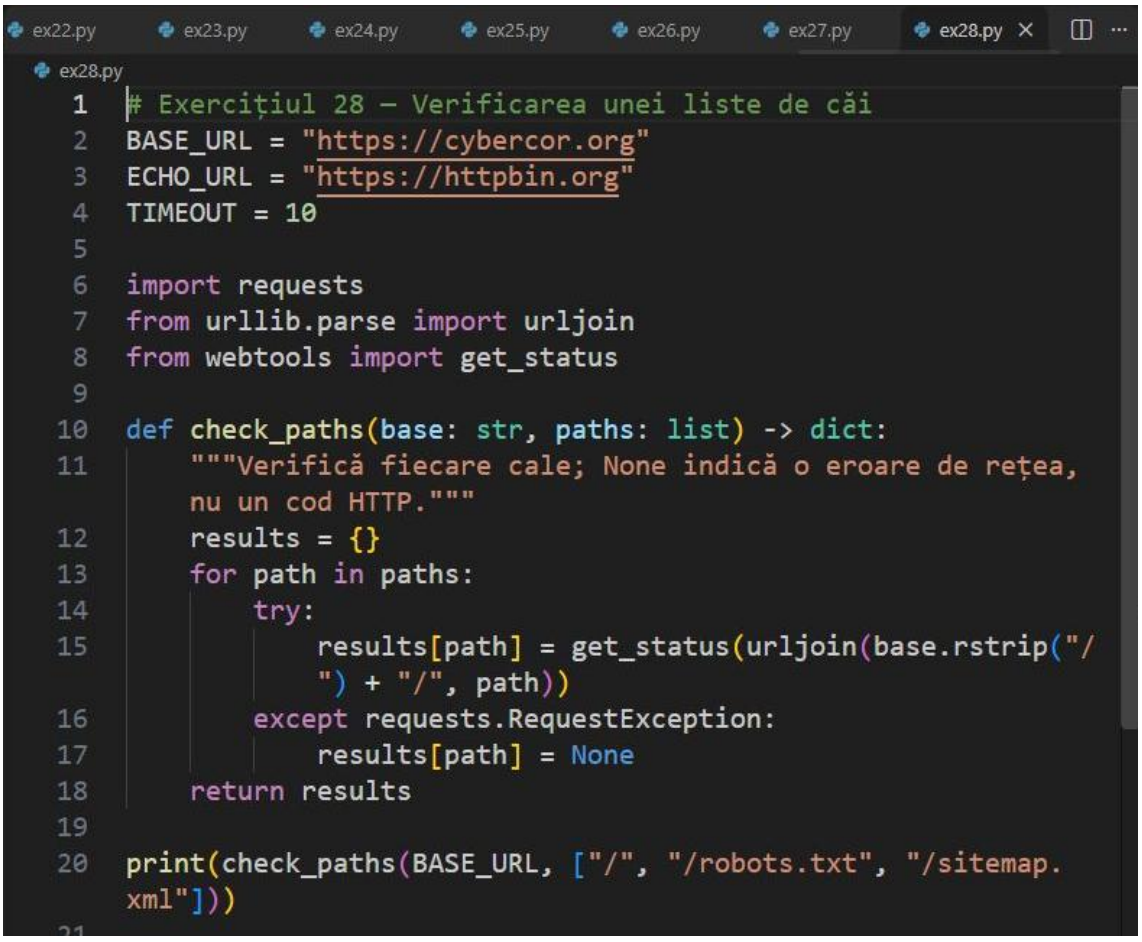



*Fig. 1.28. Codul exercițiului 28 în Visual Studio Code*

**Rezultatul rulării**

```
{'/': 200, '/robots.txt': 200, '/sitemap.xml': 200}
```

### Exercițiul 29 Argumente cu nume

Am definit o valoare implicită și am apelat funcția cu argumentul numit name.

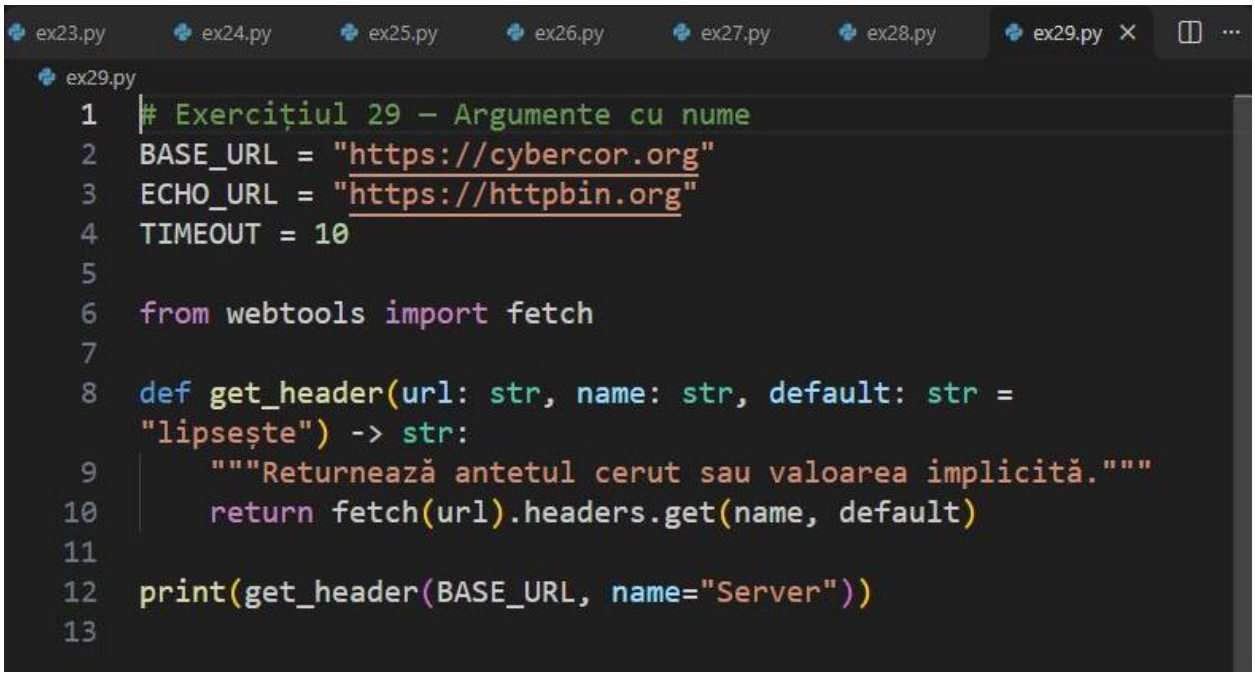



*Fig. 1.29. Codul exercițiului 29 în Visual Studio Code*

**Rezultatul rulării**

```
cloudflare
```

### Exercițiul 30 Antete de securitate

Am verificat doar prezența celor cinci antete indicate, fără să interpretez scorul ca evaluare completă de securitate.

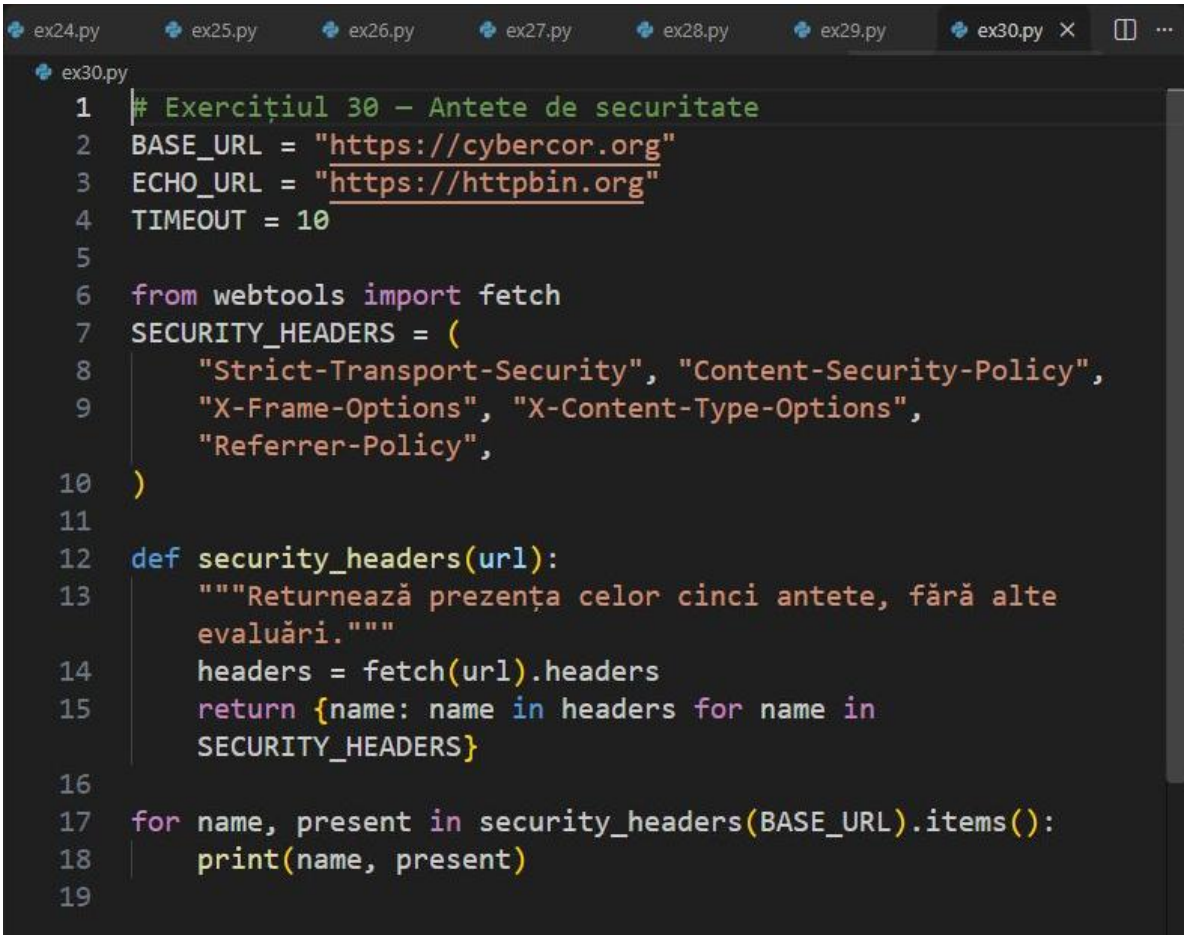



***Fig. 1.30. Codul exercițiului 30 în Visual Studio Code***

**Rezultatul rulării**

```
Strict-Transport-Security False
Content-Security-Policy False
X-Frame-Options True
X-Content-Type-Options True
Referrer-Policy True
```

### Exercițiul 31 Funcții care apelează funcții

Am compus funcția de verificare a antetelor cu funcția care calculează scorul.

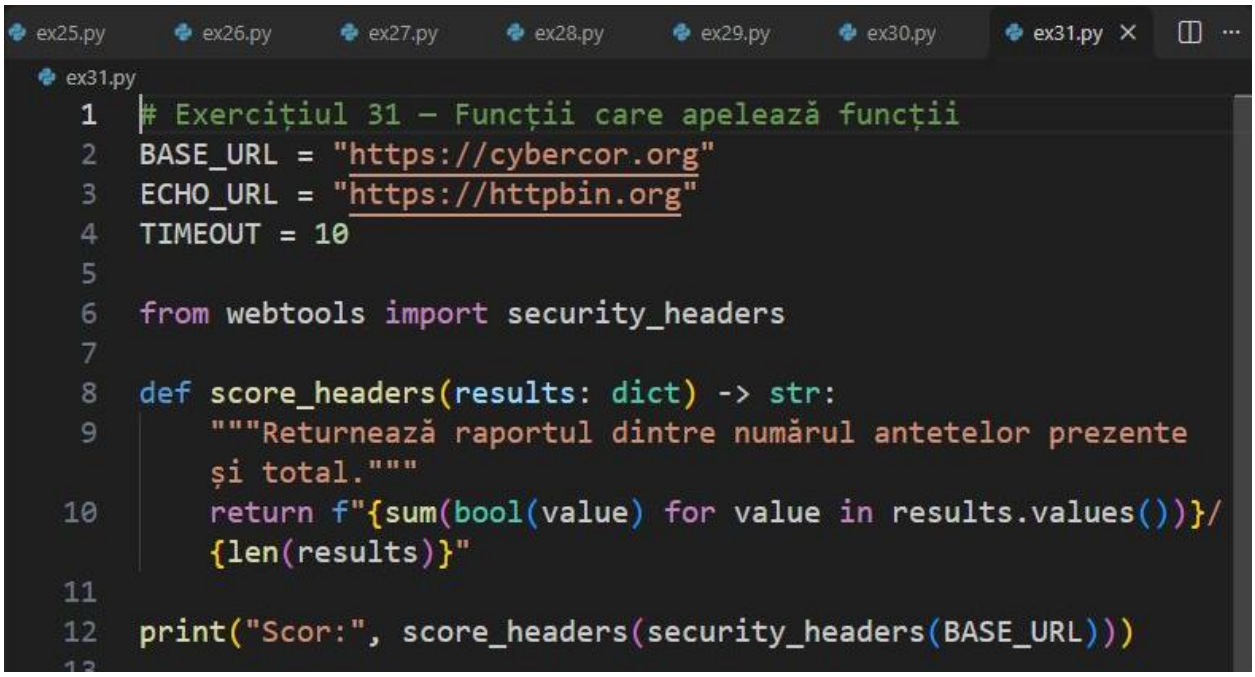



***Fig. 1.31. Codul exercițiului 31 în Visual Studio Code***

**Rezultatul rulării**

```
Scor: 3/5
```

### Exercițiul 32 Fișierul robots și directivele Disallow

Am separat citirea robots.txt de analiza directivelor. Valorile vide și comentariile nu produc căi fictive.

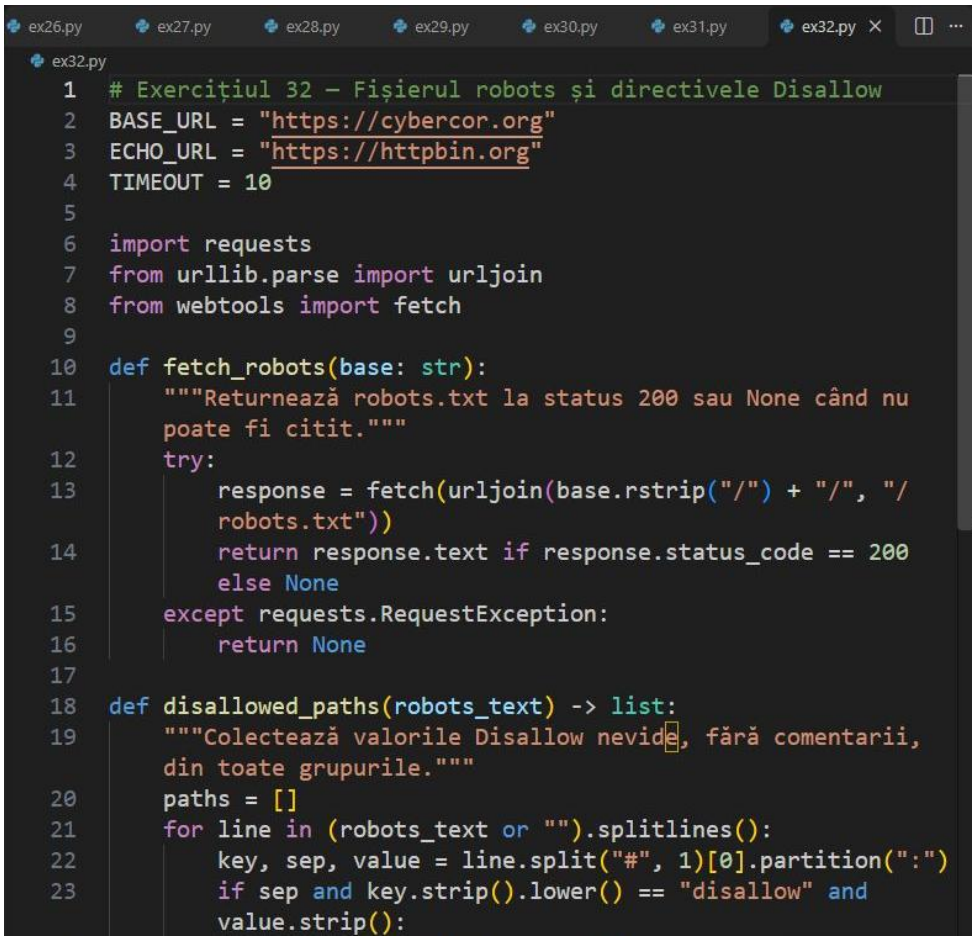



***Fig. 1.32. Codul exercițiului 32 în Visual Studio Code***

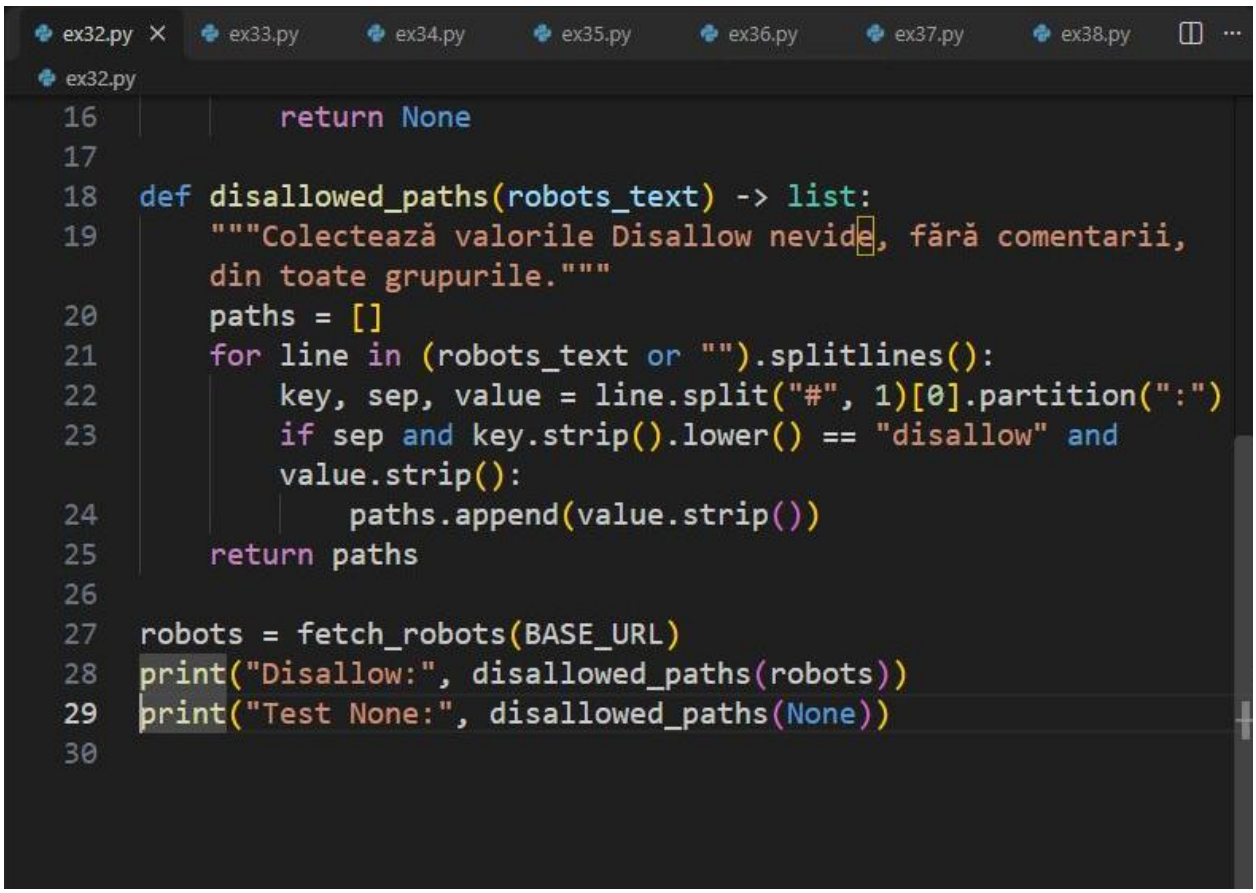



***Fig. 1.32a. Continuarea implementării exercițiului 32***

**Rezultatul rulării**

```
Disallow: []
Test None: []
```

### Exercițiul 33 Număr variabil de argumente

Am folosit *urls pentru a primi oricâte URL-uri și a returna timpii Requests elapsed.

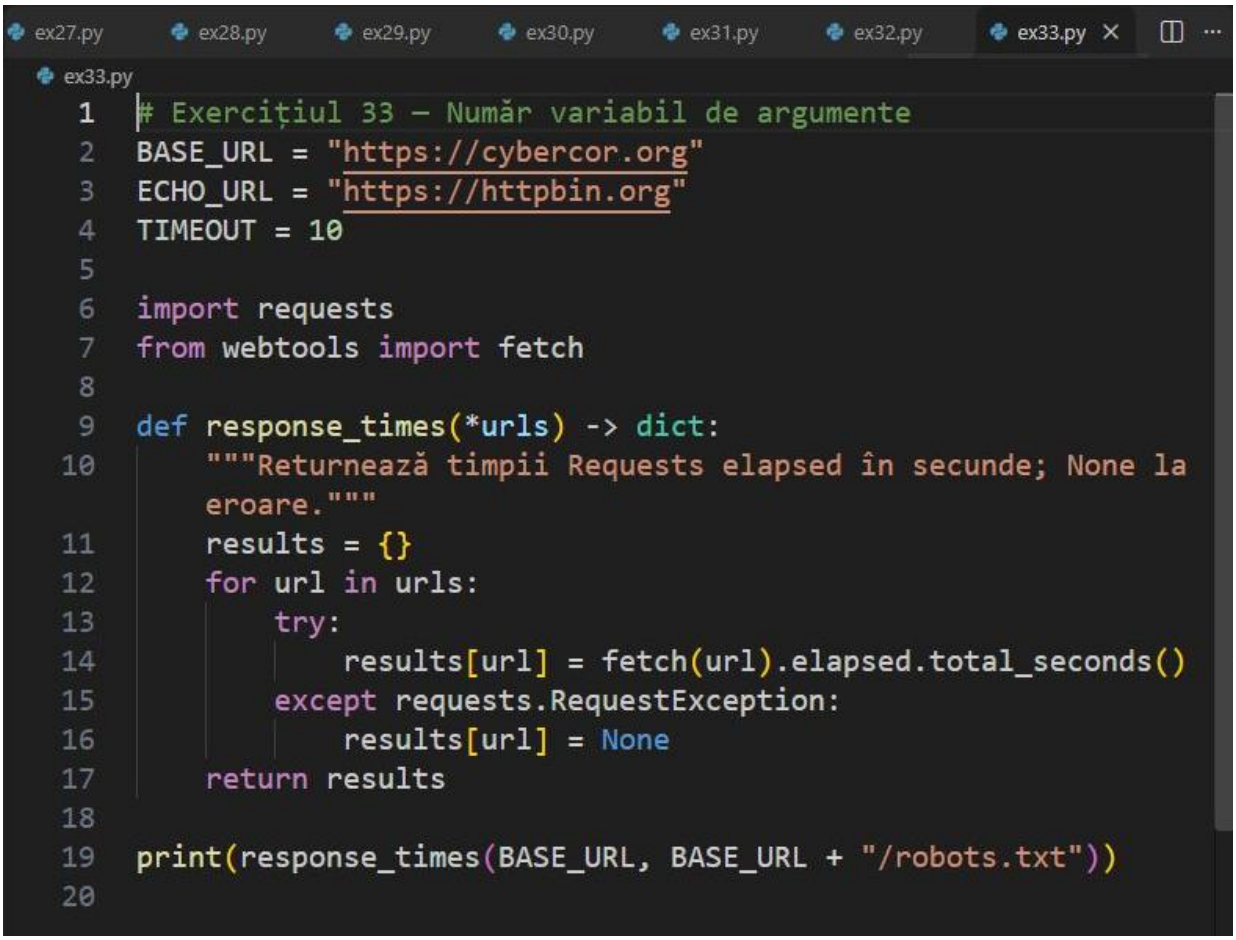



*Fig. 1.33. Codul exercițiului 33 în Visual Studio Code*

**Rezultatul rulării**

```
{'https://cybercor.org': 0.249707, 'https://cybercor.org/robots.txt': 0.176209}
```

### Exercițiul 34 Argumente numite colectate

Am folosit **details pentru detalii cu nume și le-am prezentat în ordinea primită.

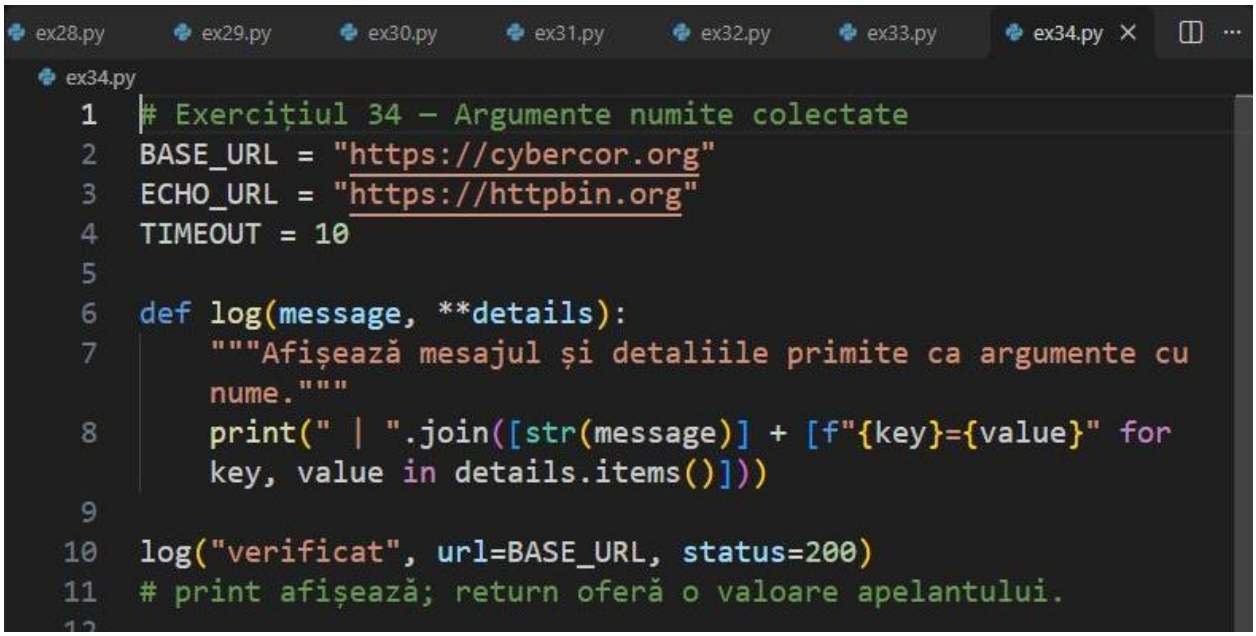



***Fig. 1.34. Codul exercițiului 34 în Visual Studio Code***

**Rezultatul rulării**

```
verificat | url=https://cybercor.org | status=200
```

# Partea 4 Module standard

### Exercițiul 35 Descompunerea URL-ului

Am descompus URL-ul fără să accesez resursa respectivă.

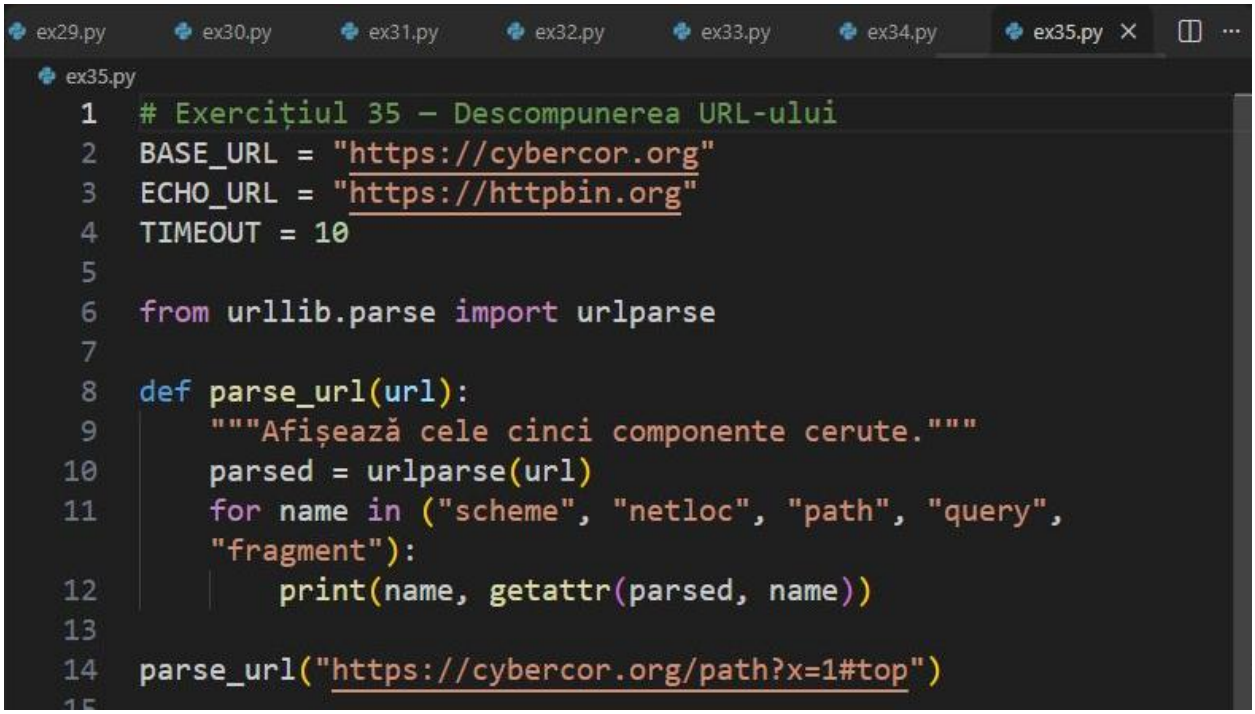



*Fig. 1.35. Codul exercițiului 35 în Visual Studio Code*

**Rezultatul rulării**

```
scheme https
netloc cybercor.org
path /path
query x=1
fragment top
```

### Exercițiul 36 Compunerea URL-urilor

Am folosit urljoin pentru căi absolute, relative și cu traversare spre directorul părinte.

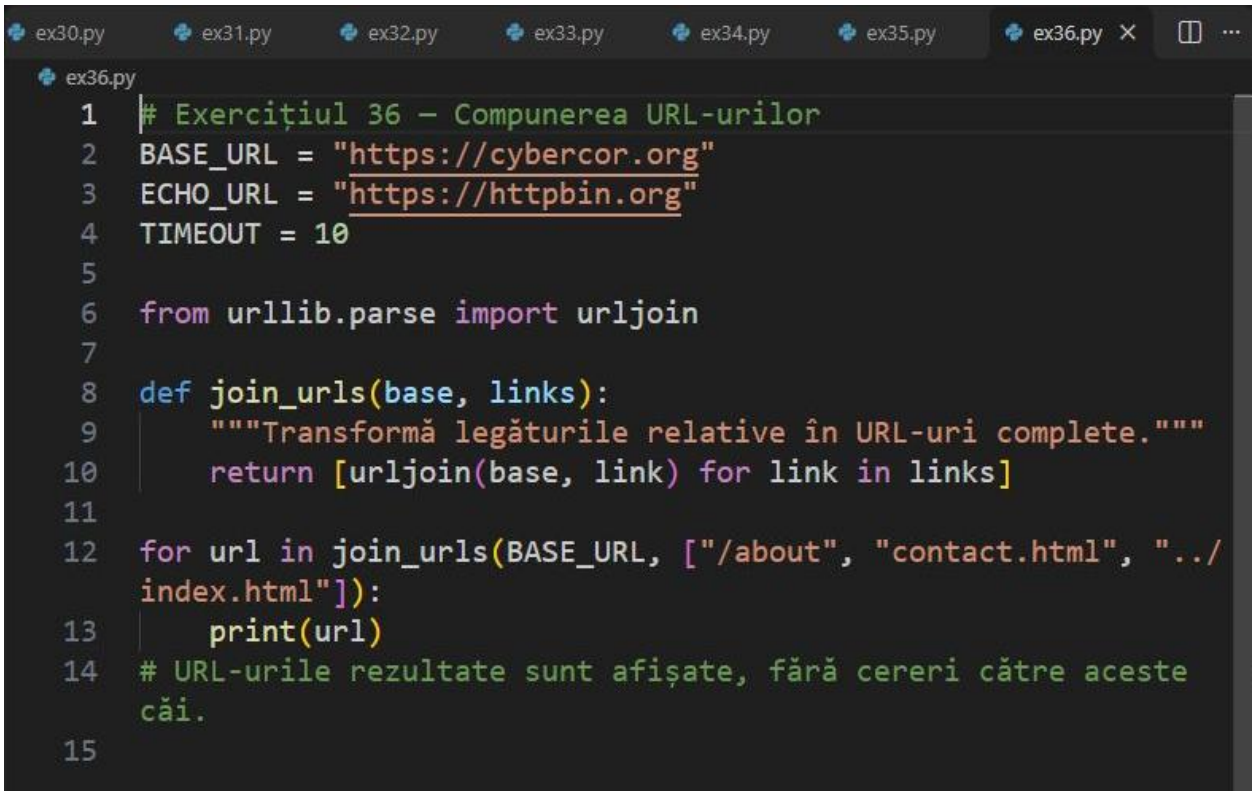



*Fig. 1.36. Codul exercițiului 36 în Visual Studio Code*

**Rezultatul rulării**

```
https://cybercor.org/about
https://cybercor.org/contact.html
https://cybercor.org/index.html
```

### Exercițiul 37 Extragerea legăturilor fără duplicate

Am folosit expresia regulată cerută și am păstrat ordinea la eliminarea duplicatelor.

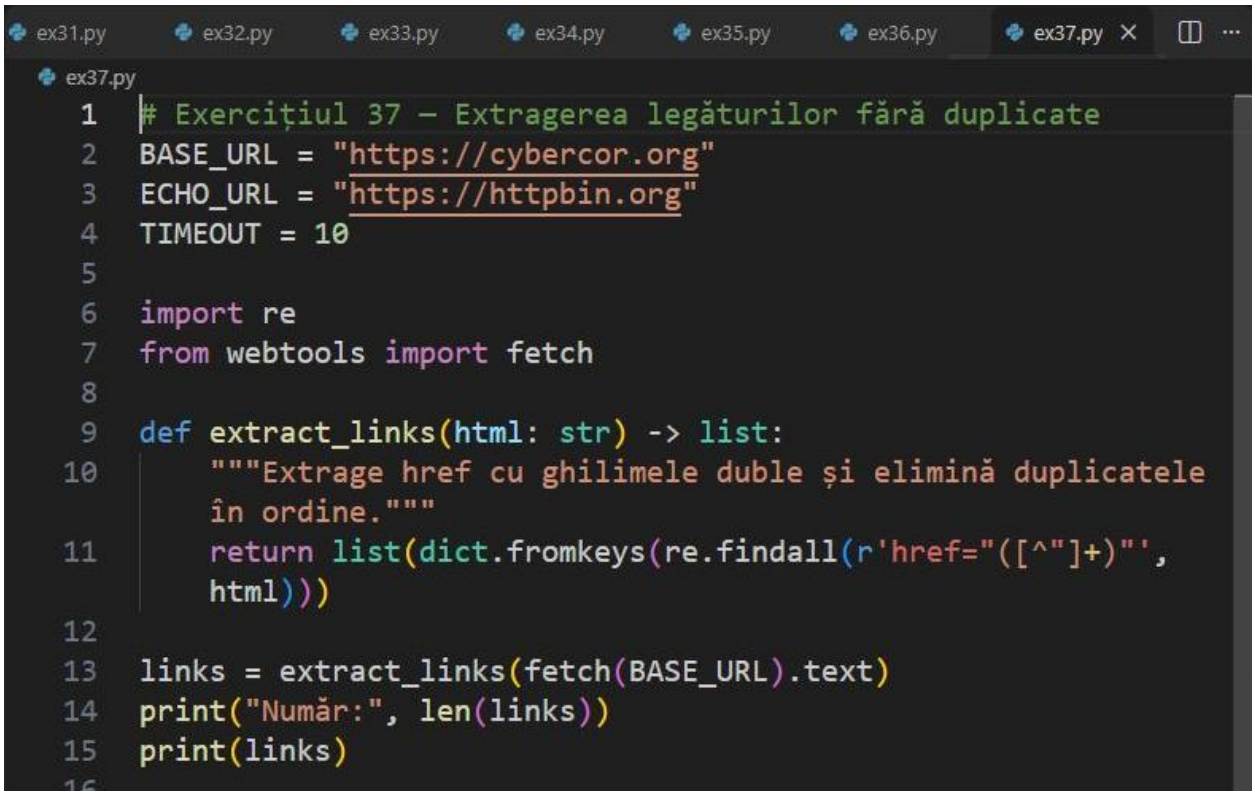



*Fig. 1.37. Codul exercițiului 37 în Visual Studio Code*

**Rezultatul rulării**

```
Număr: 1
['/assets/index-DlIqjleG.css']
```

### Exercițiul 38 Legături interne și externe

Am compus URL-urile și am comparat domeniile. Schemele mailto, tel și javascript nu sunt numărate ca pagini HTTP.

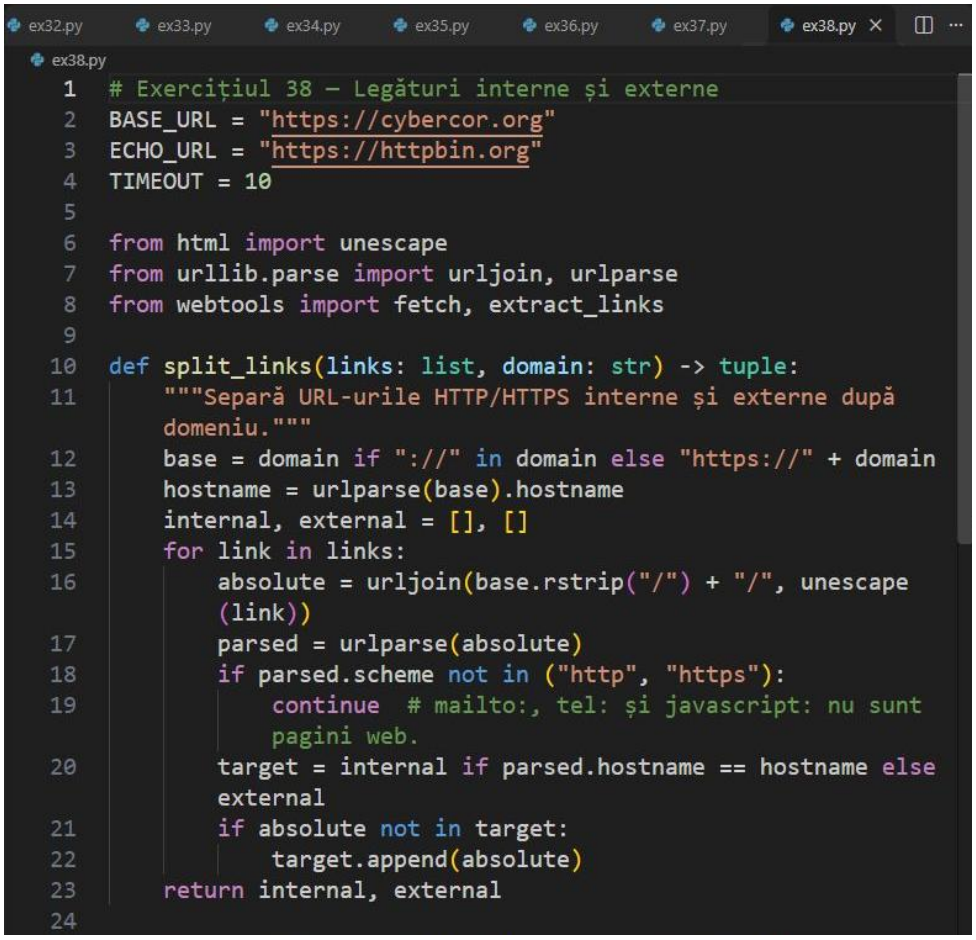



***Fig. 1.38. Codul exercițiului 38 în Visual Studio Code***

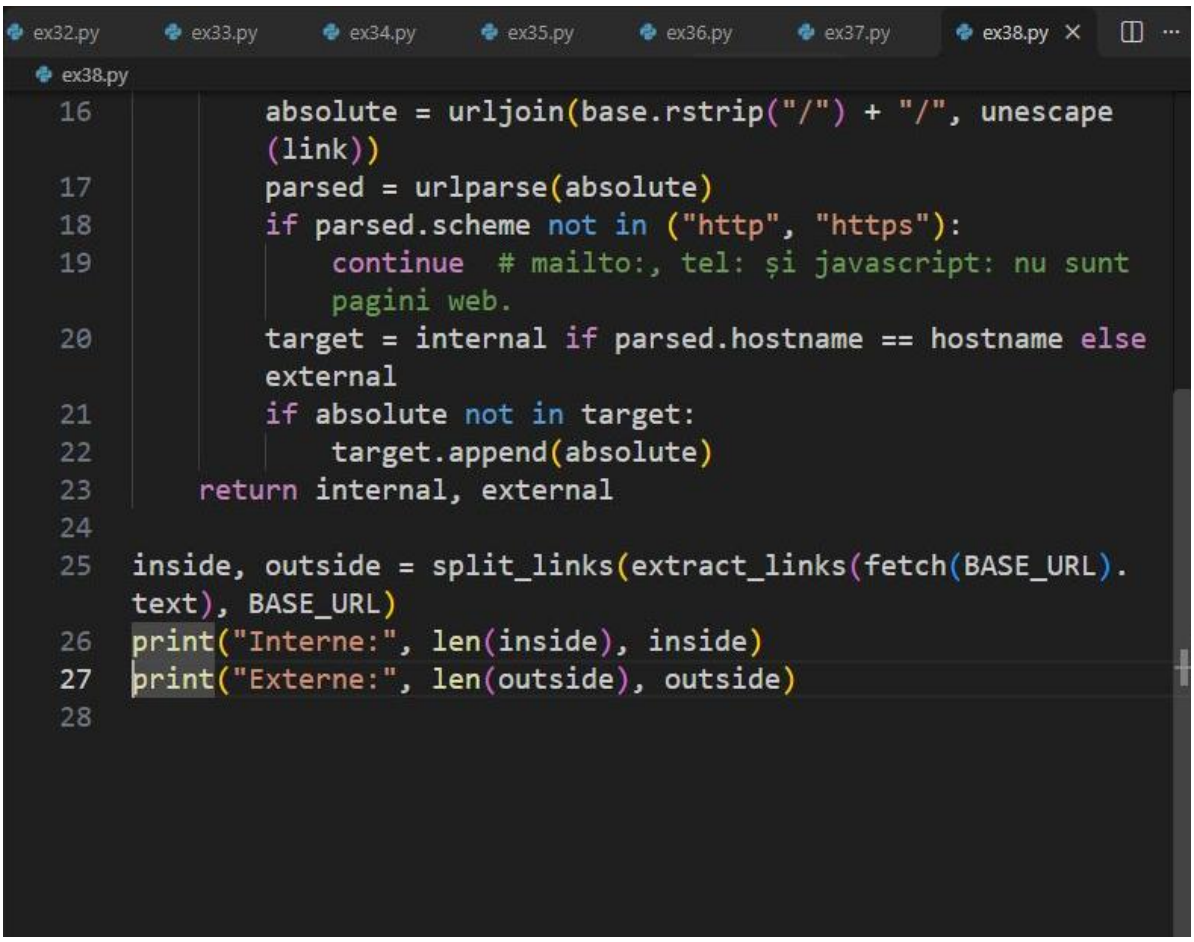



***Fig. 1.38a. Continuarea implementării exercițiului 38***

**Rezultatul rulării**

```
Interne: 1 ['https://cybercor.org/assets/index-DlIqjleG.css']
Externe: 0 []
```

### Exercițiul 39 Parserul de imagini

Am verificat parserul mai întâi pe HTML cunoscut, apoi pe pagina reală. Testul complet din enunț există și în part4_modules.py.

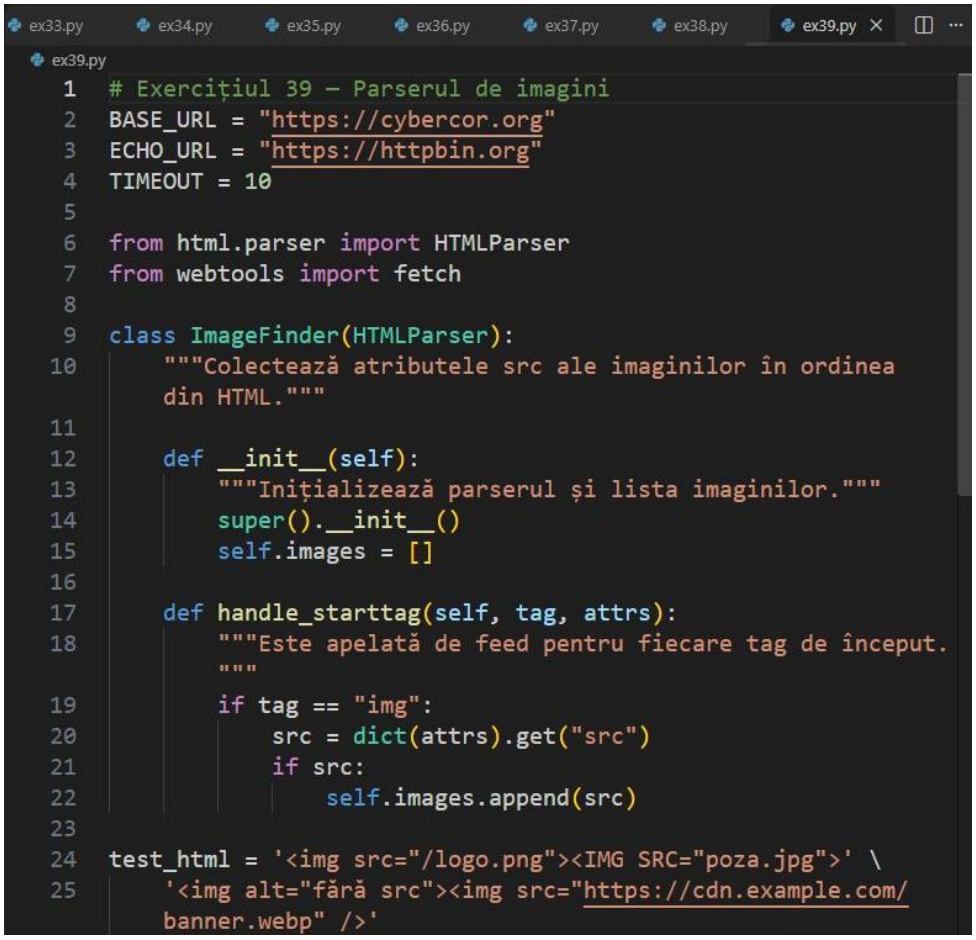



***Fig. 1.39. Codul exercițiului 39 în Visual Studio Code***

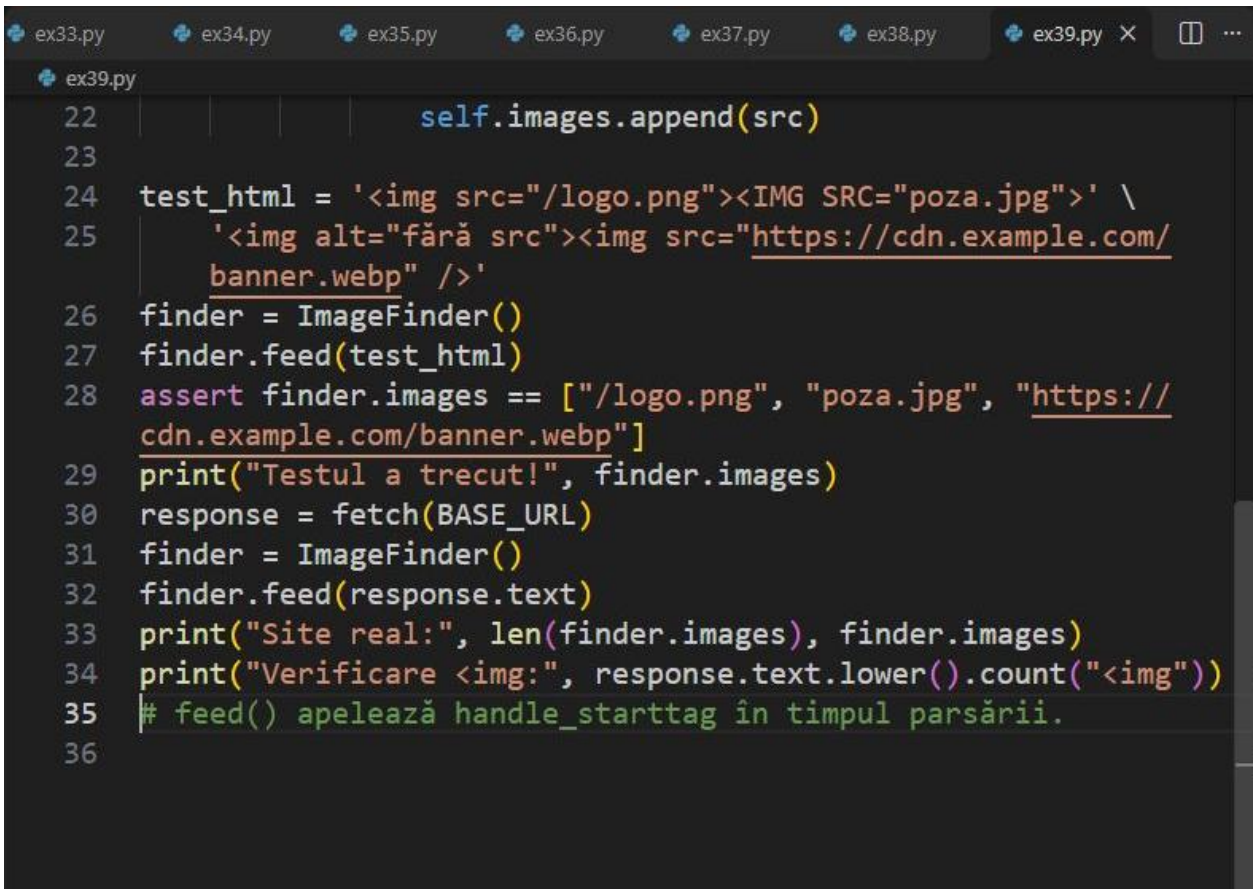



***Fig. 1.39a. Continuarea implementării exercițiului 39***

**Rezultatul rulării**

```
Testul a trecut! ['/logo.png', 'poza.jpg', 'https://cdn.example.com/banner.webp']
Site real: 0 []
Verificare <img: 0
```

Testul local a trecut, iar count("<img") confirmă absența tag-urilor img în HTML-ul primit de program. Lista goală pentru site nu indică o eroare a parserului.

### Exercițiul 40 Amprenta SHA256 a paginii

Am calculat amprenta corpului primit la două descărcări și am comparat rezultatele.

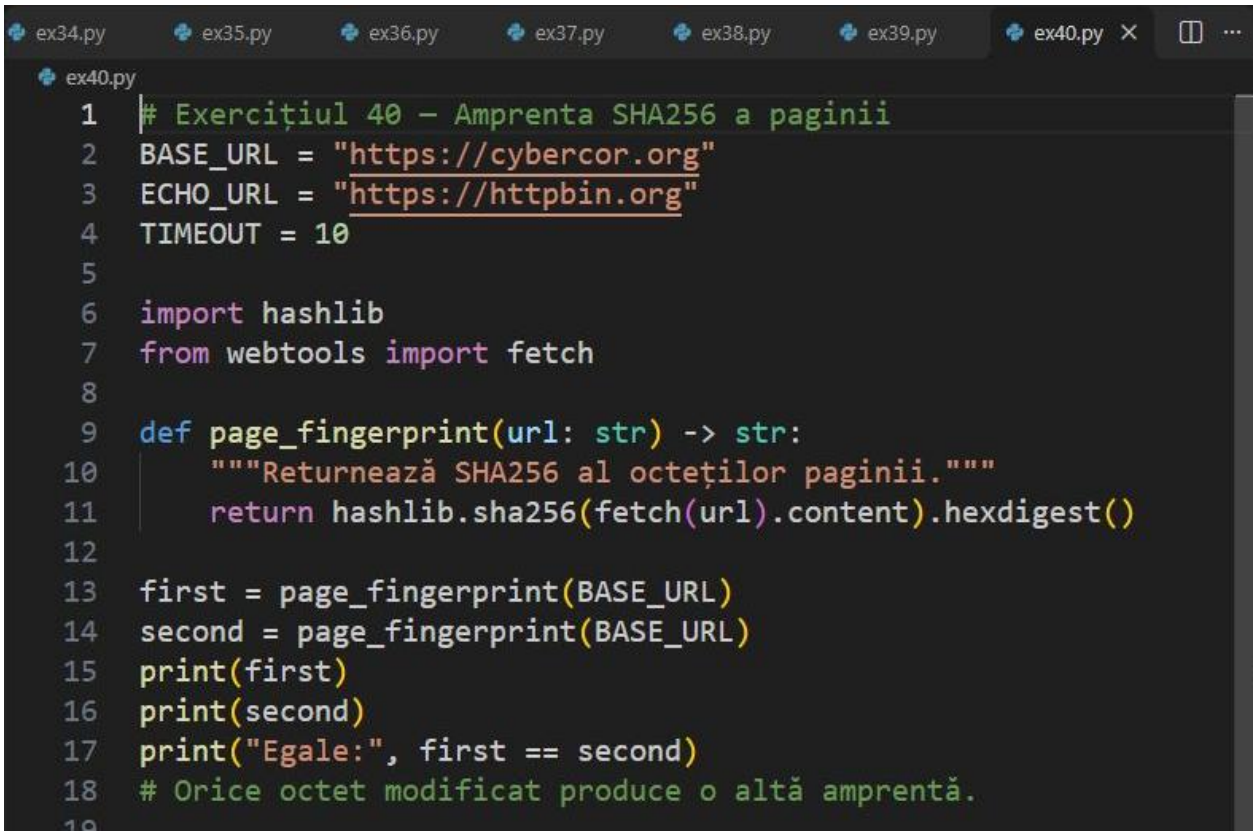



***Fig. 1.40. Codul exercițiului 40 în Visual Studio Code***

**Rezultatul rulării**

```
c04dd60a2651465e1aa75da82b7e1c5daf04b38c3663c6aaef38efb6e4be89e4
c04dd60a2651465e1aa75da82b7e1c5daf04b38c3663c6aaef38efb6e4be89e4
Egale: True
```

### Exercițiul 41 Salvarea și încărcarea JSON

Am convertit antetele în dict, le-am serializat și am citit înapoi antetul Content-Type.

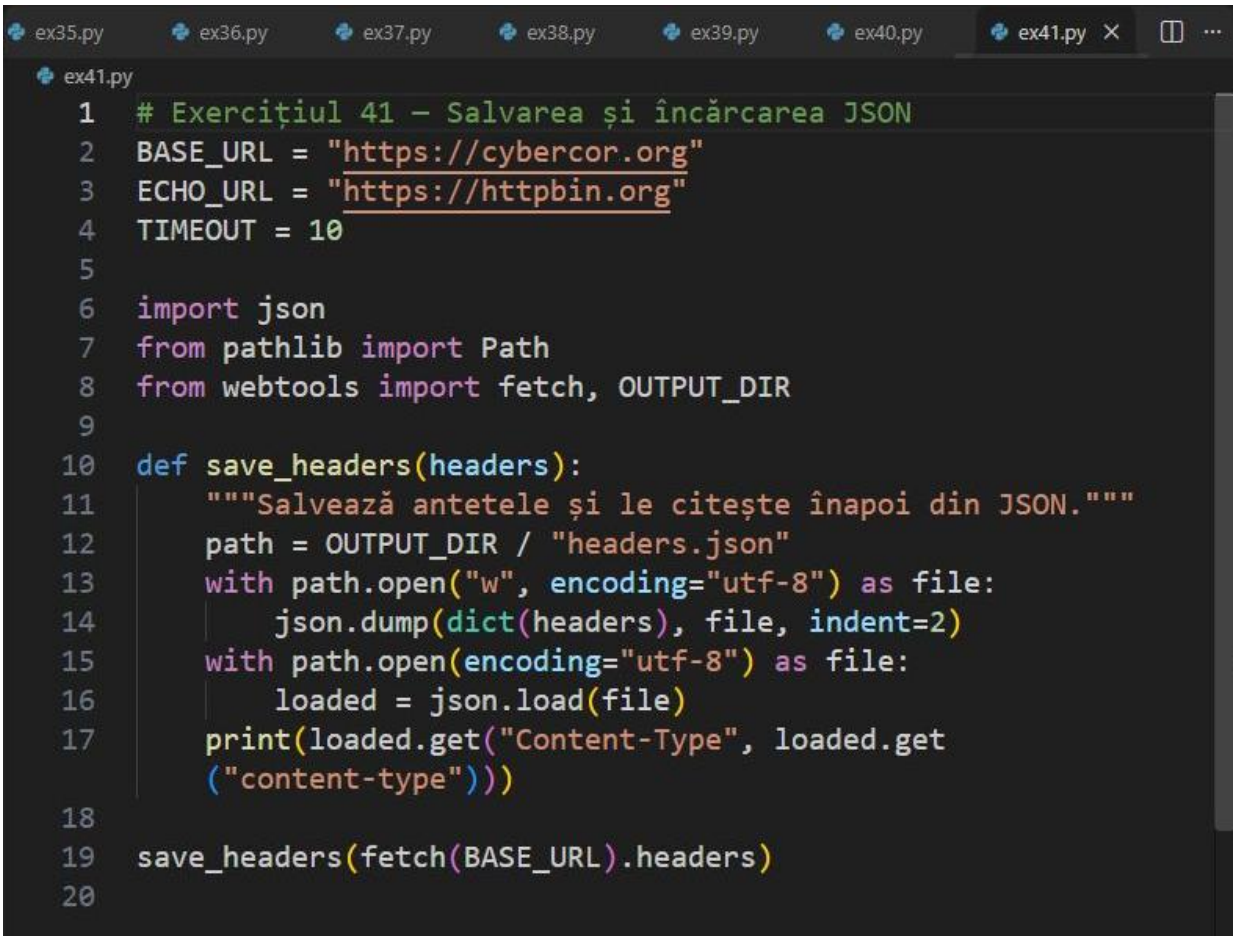



***Fig. 1.41. Codul exercițiului 41 în Visual Studio Code***

**Rezultatul rulării**

```
text/html; charset=utf-8
```

### Exercițiul 42 Rezolvarea DNS

Am interogat DNS pentru o adresă IPv4 a domeniului cybercor.org.

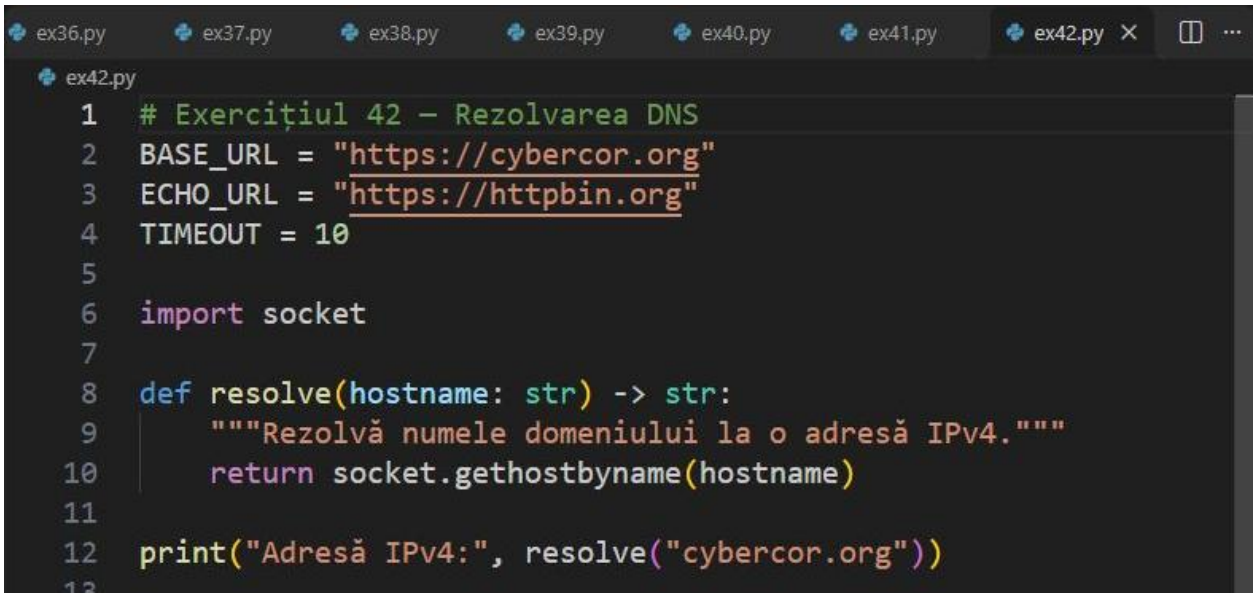



***Fig. 1.42. Codul exercițiului 42 în Visual Studio Code***

**Rezultatul rulării**

```
Adresă IPv4: 104.21.91.71
```

### Exercițiul 43 Zile până la expirarea certificatului

Am deschis o conexiune TLS validată, am citit notAfter și am calculat numărul de zile întregi rămase.

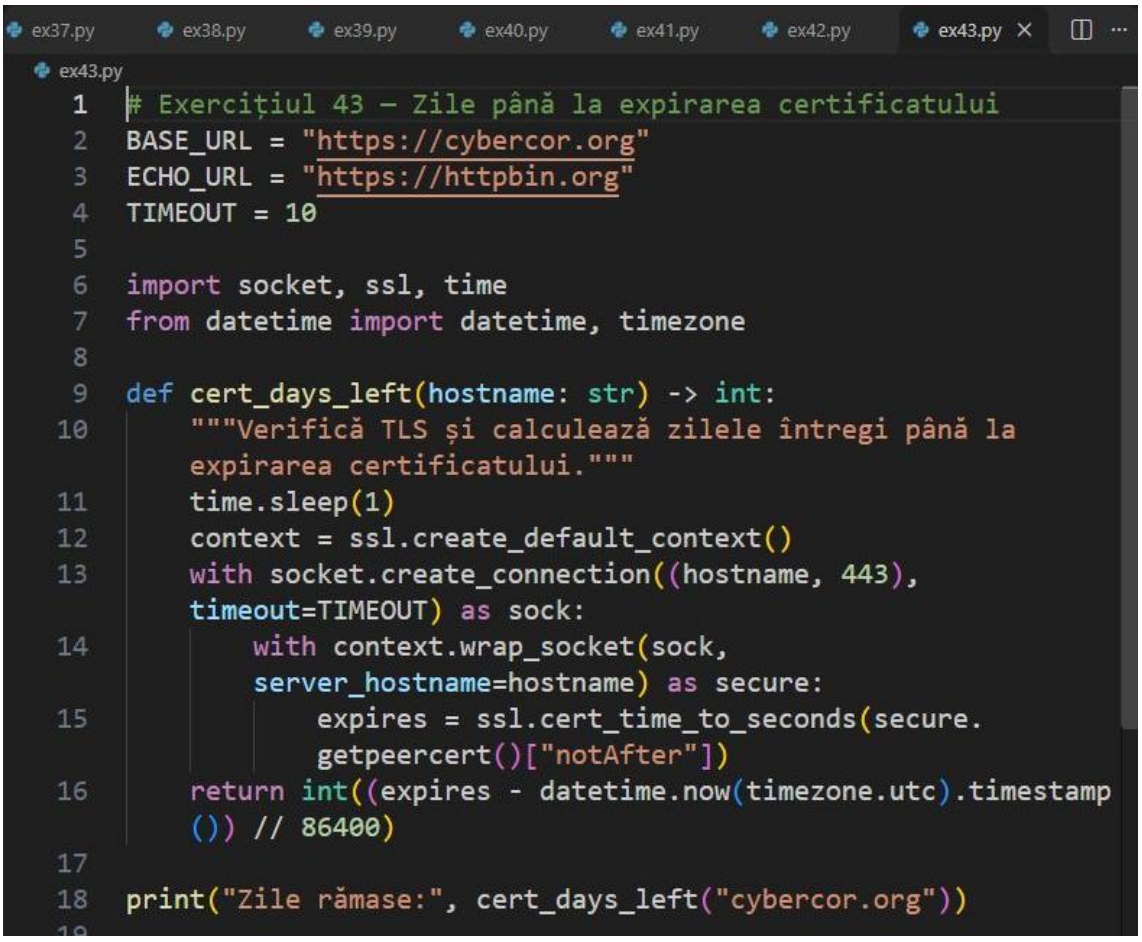



*Fig. 1.43. Codul exercițiului 43 în Visual Studio Code*

**Rezultatul rulării**

```
Zile rămase: 51
```

# Partea 5 Modul propriu și proiect final

### Exercițiul 44 Crearea modulului webtools

Am importat modulul propriu și am apelat funcțiile cu numele calificat webtools.

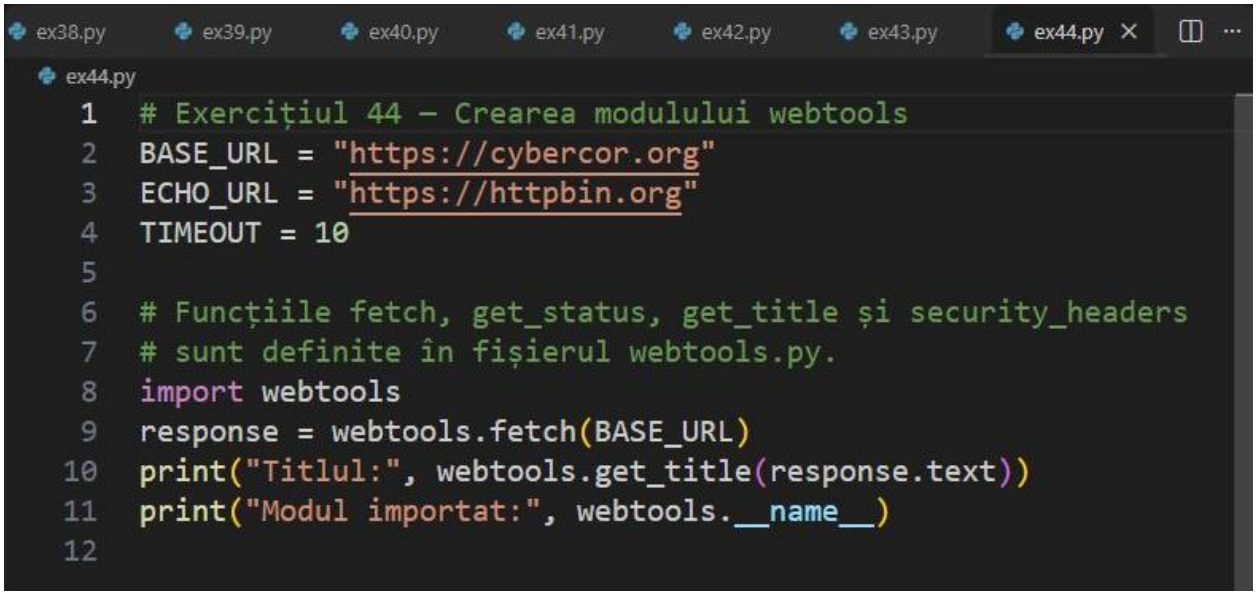



***Fig. 1.44. Codul exercițiului 44 în Visual Studio Code***

**Rezultatul rulării**

```
Titlul: CYBERCOR
Modul importat: webtools
```

### Exercițiul 45 Importul unor nume din modul

Am importat anumite funcții și am explicat riscul înlocuirii unui nume importat.

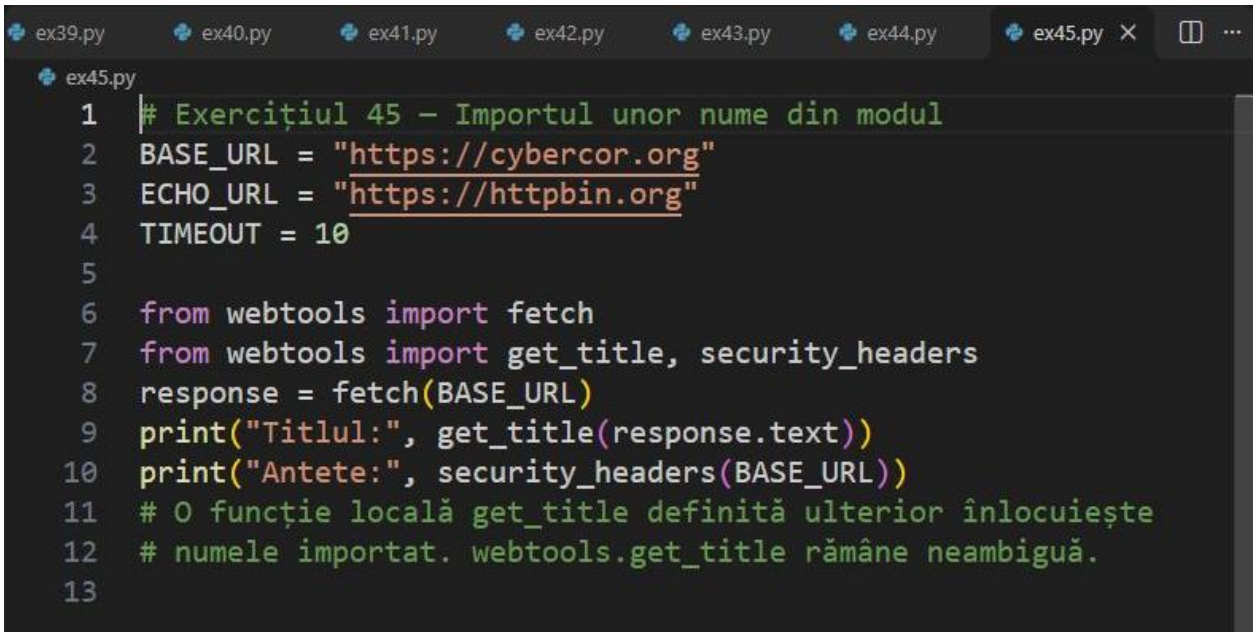



***Fig. 1.45. Codul exercițiului 45 în Visual Studio Code***

**Rezultatul rulării**

```
Titlul: CYBERCOR
Antete: {'Strict-Transport-Security': False, 'Content-Security-Policy': False,
'X-Frame-Options': True, 'X-Content-Type-Options': True, 'Referrer-Policy': True}
```

### Exercițiul 46 Blocul de execuție main

Am comparat importul modulului cu execuția lui ca __main__. Autotestul rulează doar în al doilea caz.

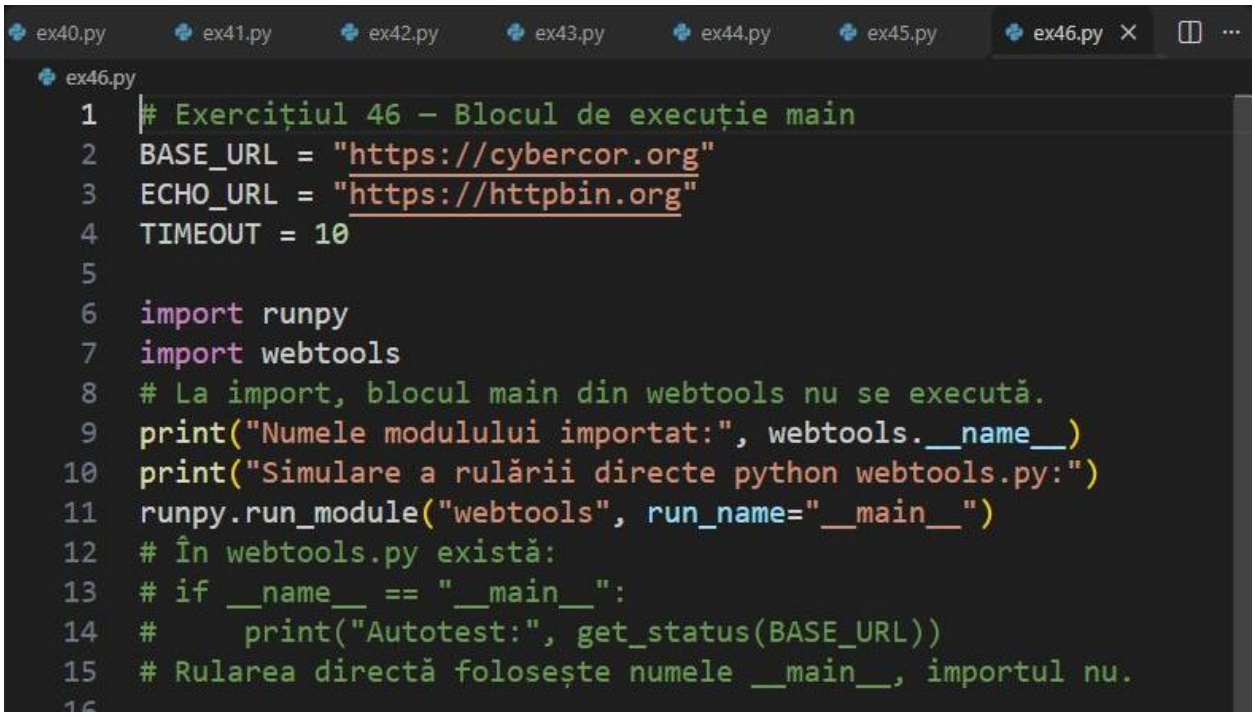



***Fig. 1.46. Codul exercițiului 46 în Visual Studio Code***

**Rezultatul rulării**

```
Numele modulului importat: webtools
Simulare a rulării directe python webtools.py:
Autotest: 200
```

### Exercițiul 47 Constantele de modul

Am importat constanta DEFAULT_HEADERS și am verificat folosirea ei pe serviciul httpbin.

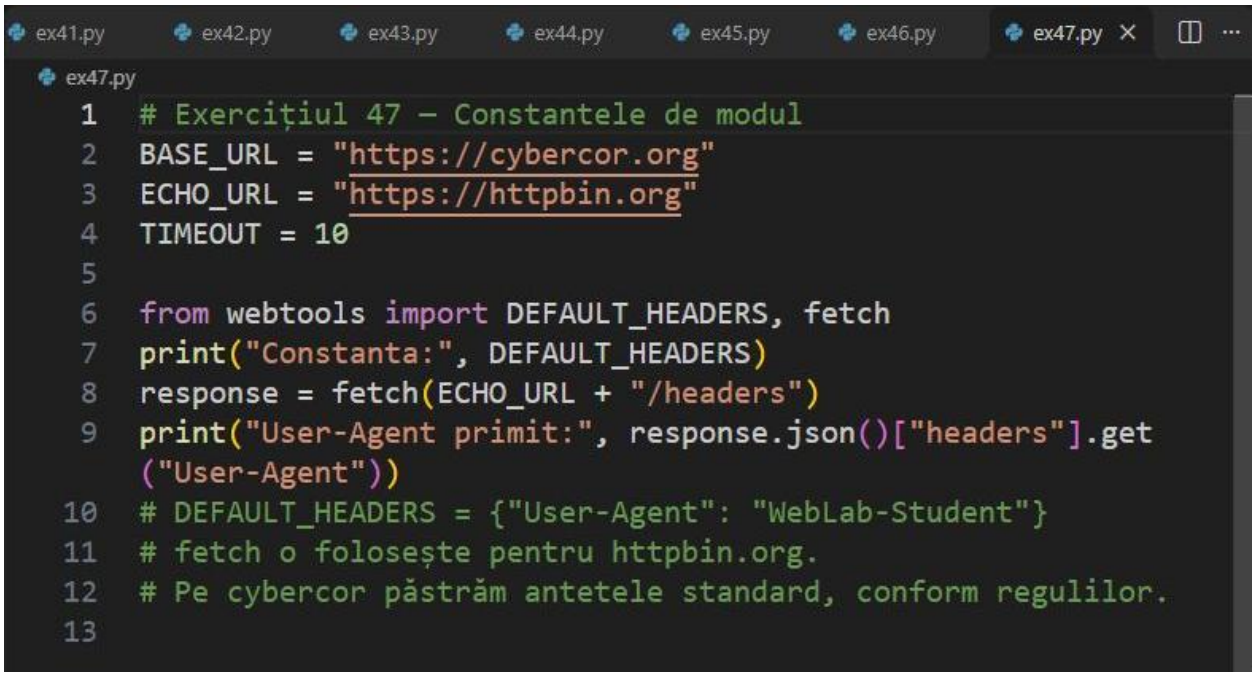



***Fig. 1.47. Codul exercițiului 47 în Visual Studio Code***

**Rezultatul rulării**

```
Constanta: {'User-Agent': 'WebLab-Student'}
User-Agent primit: WebLab-Student
```

### Exercițiul 48 Argumente din linia de comandă

Am folosit argparse pentru a prelua URL-ul din CLI și a-l transmite funcțiilor.

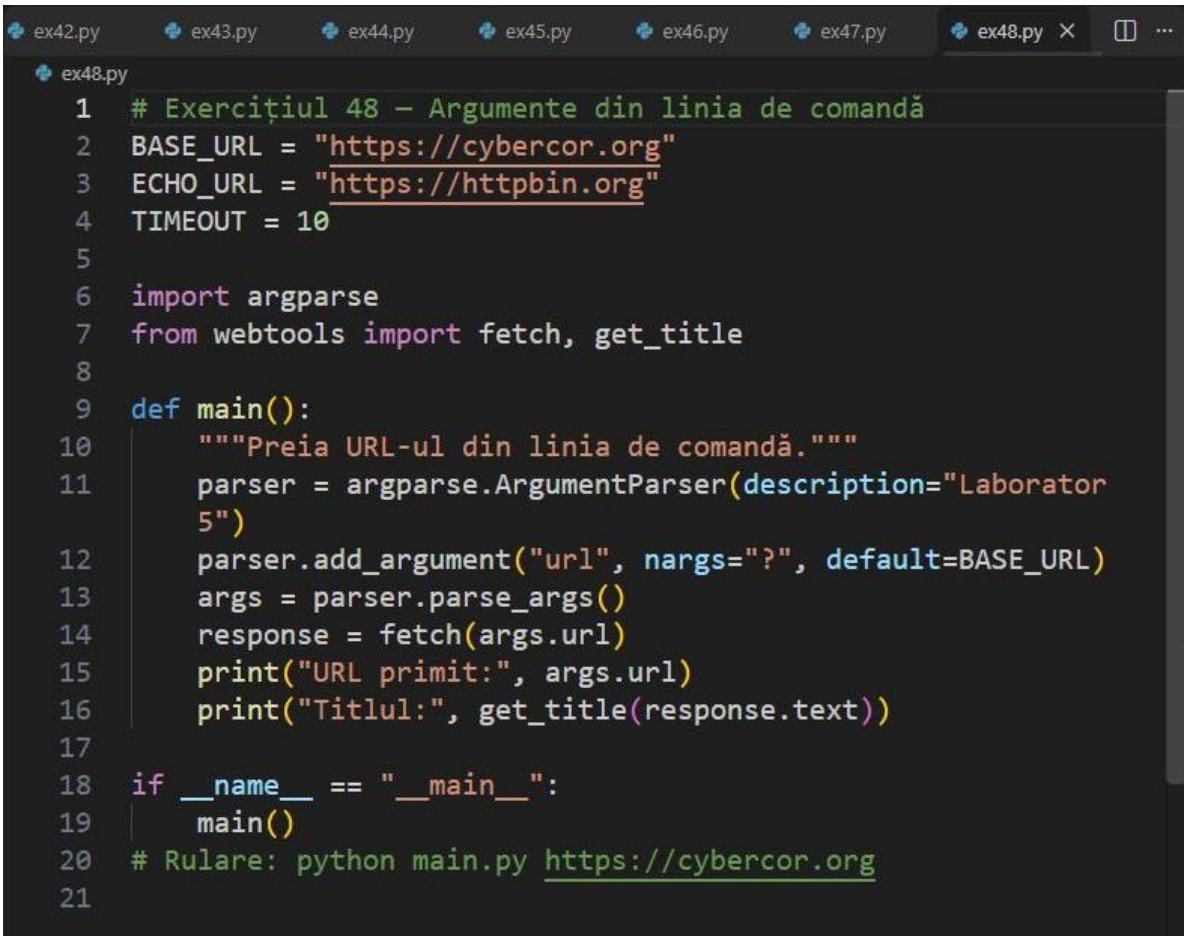



*Fig. 1.48. Codul exercițiului 48 în Visual Studio Code*

**Rezultatul rulării**

```
URL primit: https://cybercor.org
Titlul: CYBERCOR
```

### Exercițiul 49 Raportul CSV

Am salvat câte un rând pentru fiecare cale, cu status și data verificării în format ISO.

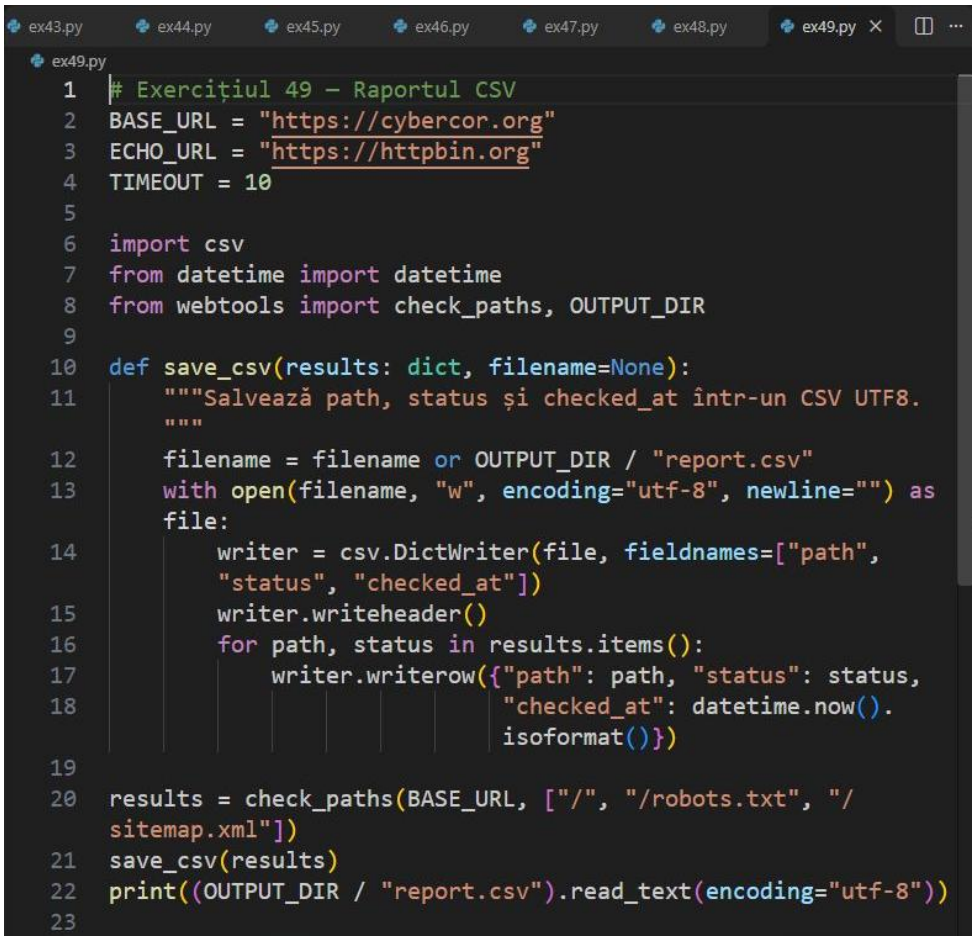



***Fig. 1.49. Codul exercițiului 49 în Visual Studio Code***

**Rezultatul rulării**

```
path,status,checked_at
/,200,2026-10-13T15:29:37.776175
/robots.txt,200,2026-10-13T15:29:37.776210
/sitemap.xml,200,2026-10-13T15:29:37.776218
```

### Exercițiul 50 Proiectul final site_report

Am compus funcțiile într-un raport complet, l-am afișat și l-am salvat în JSON. Capturi suplimentare prezintă implementarea site_report din modulul propriu.

ex44.py ex45.py ex46.py ex47.py ex48.py ex49.py ex50.py

ex50.py

```
1  # Exercițiul 50 – Proiectul final site_report
2  BASE_URL = "https://cybercor.org"
3  ECHO_URL = "https://httpbin.org"
4  TIMEOUT = 10
5
6  from webtools import site_report, OUTPUT_DIR
7  import json
8  # Implementarea completă se află în webtools.py.
9  # Funcția colectează:
10 #   status și URL final, titlu, IP, redirecturi HTTP,
11 #   scor antete, expirare TLS, href interne/externe,
12 #   directive Disallow din robots.txt.
13 report = site_report(BASE_URL)
14 with (OUTPUT_DIR / "report.json").open(encoding="utf-8") as
   file:
15     saved = json.load(file)
16 print("Fișier JSON valid:", saved == report)
17 print("Raport salvat:", OUTPUT_DIR / "report.json")
```

***Fig. 1.50. Codul exercițiului 50 în Visual Studio Code***

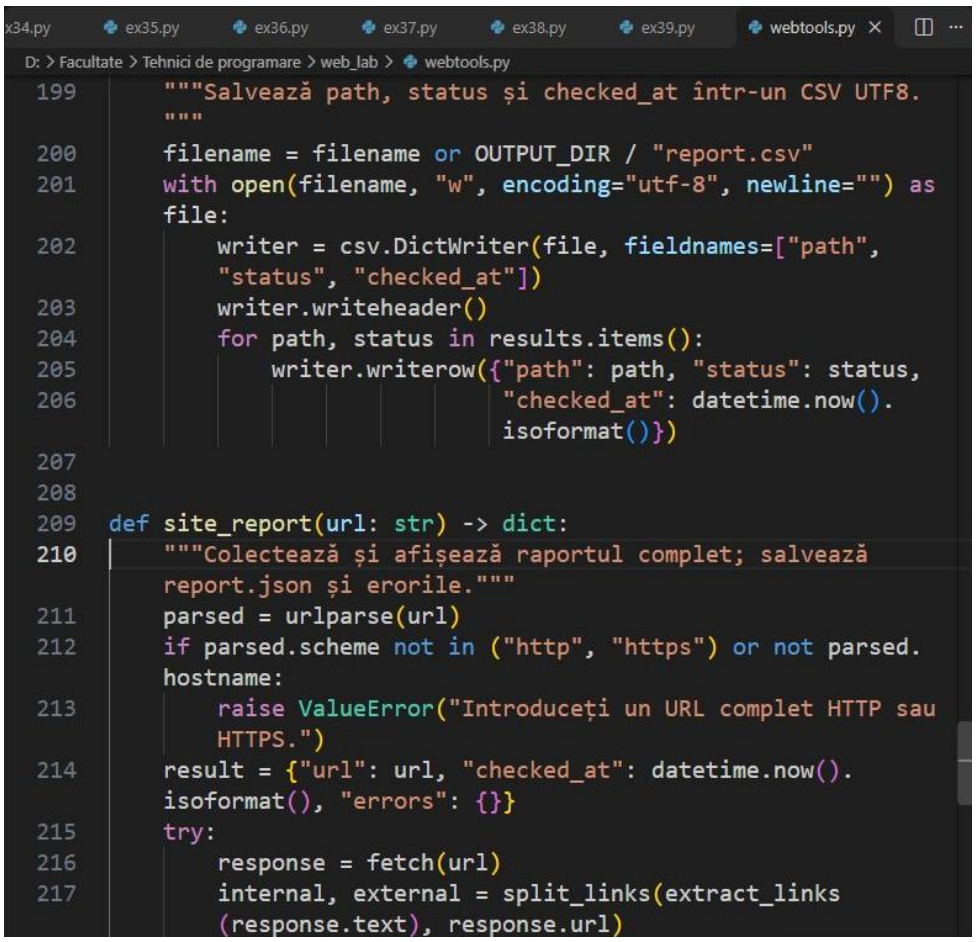



***Fig. 1.50a. Continuarea implementării exercițiului 50***

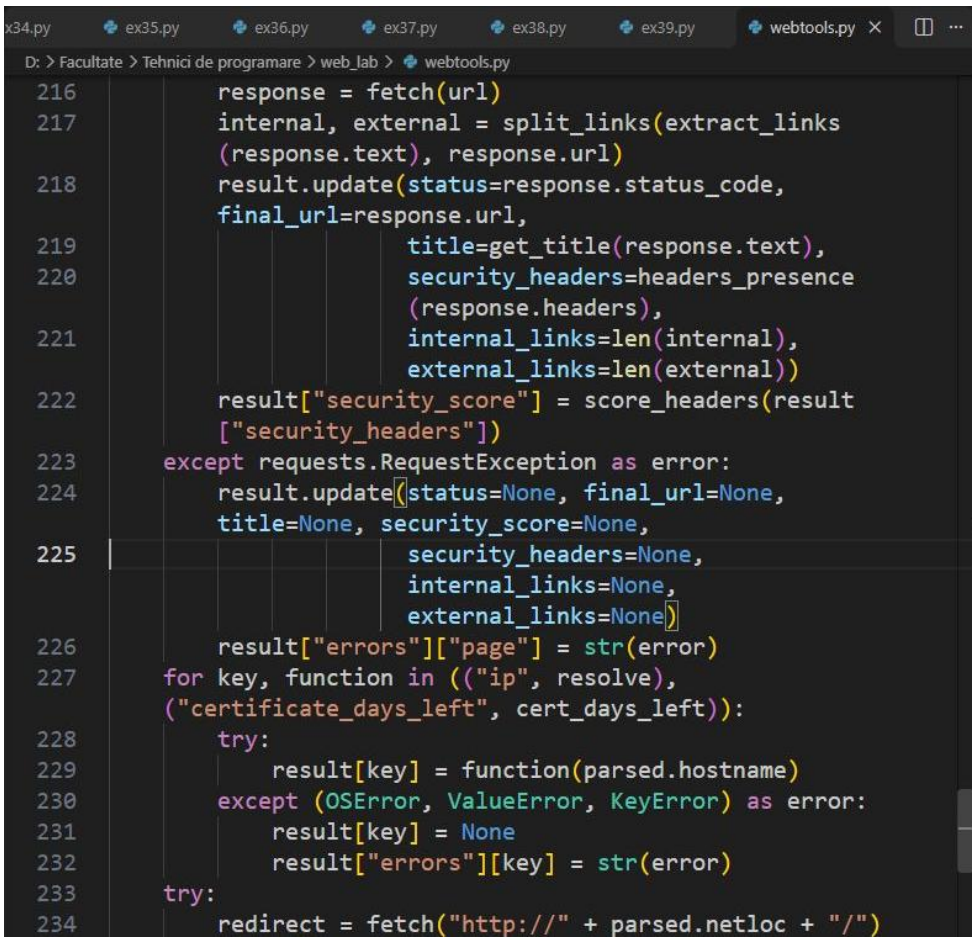



***Fig. 1.50b. Continuarea implementării exercițiului 50***

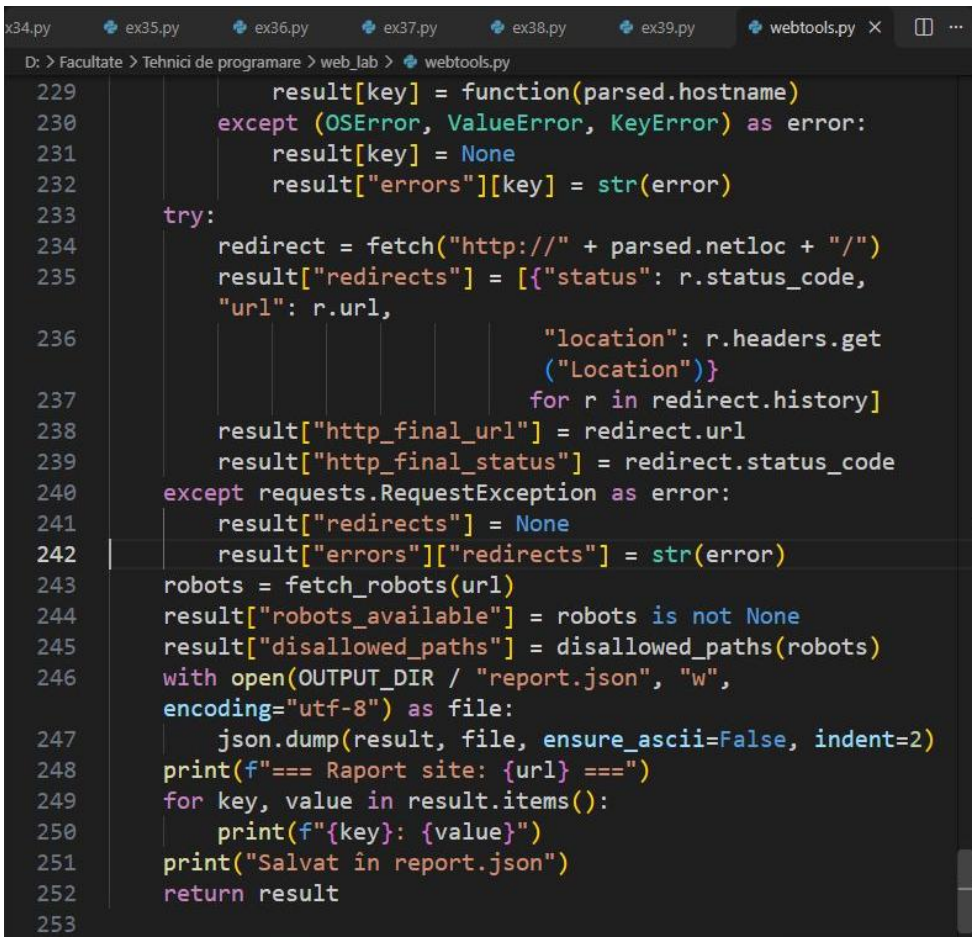



***Fig. 1.50c. Continuarea implementării exercițiului 50***

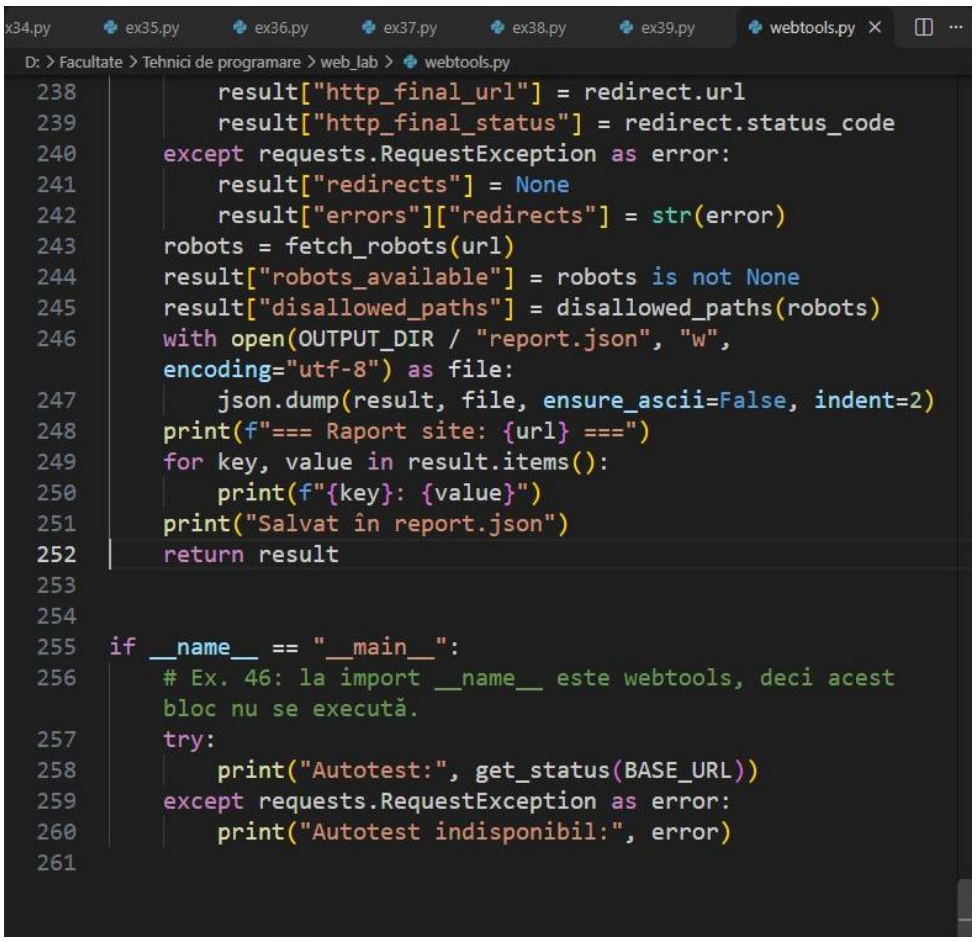



*Fig. 1.50d. Continuarea implementării exercițiului 50*

**Rezultatul rulării**

```
status: 200
final_url: https://cybercor.org/
title: CYBERCOR
internal_links: 1
external_links: 0
security_score: 3/5
ip: 104.21.91.71
certificate_days_left: 51
disallowed_paths: []
Fișier JSON valid: True
```

# Bonus BeautifulSoup și concluzii

Bonusul implementează get_title_bs4 și extract_links_bs4 în bonus_bs4.py. BeautifulSoup recunoaște tag-uri cu atribute și href cu ghilimele simple. Aceste situații depășesc logica simplă bazată pe find sau expresia regulată cerută în laborator.

```
HTML cunoscut:  Laborator 5
Legături: [] ['/about']
Titlu find / bs4: CYBERCOR CYBERCOR
Legături regex / bs4: 1 1
```

### Verificarea soluției

Cele 50 de exerciții individuale au fost executate. Cele șapte teste locale verifică parsarea titlului, robots.txt, legăturile, antetele, imaginile, erorile de conexiune și HTTPError pentru un răspuns 404. Importul webtools nu trimite cereri; autotestul rulează numai la execuția directă.

```
Ran 7 tests
OK
```

### Concluzii

Am organizat accesul la rețea și prelucrarea răspunsurilor în funcții reutilizabile. Importurile permit folosirea modulelor standard, externe și proprii. Metodele realizează operații asupra obiectelor, iar atributele oferă valorile lor. Returnarea datelor a permis compunerea raportului final și salvarea rezultatelor în CSV și JSON.

Raportul final a identificat titlul CYBERCOR, redirecționarea HTTP spre HTTPS și trei dintre cele cinci antete verificate. În HTML-ul descărcat s-a găsit un href intern către o resursă CSS și niciun tag img. Scorul antetelor reprezintă numai prezența lor, fără a constitui o evaluare completă de securitate.

### Surse

Enunțul Laborator Python funcții metode și importuri pe web. Documentația oficială Requests Quickstart: https://requests.readthedocs.io/en/latest/user/quickstart/. Datele experimentale și capturile sunt incluse în directorul web_lab.